Dédicaces

Je dédie ce modeste travail :

A mes parents, mes anges protecteurs, qui ont fait de moi la personne que je suis aujourd'hui. Tous les dictionnaires du monde n'arrivent pas à exprimer ce que je ressens pour vous. Vous êtes le meilleur don que Dieu puisse m'offrir dans cette vie. Je vous aime fortement.

A mon frère Nawfel, un grand homme au cœur toujours ouvert, qui m'a accueilli et m'a soutenu sur tous les plans. Merci d'être là au moment où j'en avais besoin.

Ma sœur Mabrouka, qui a toujours cru en moi, m'a encouragé, pour son affection, son amour inconditionnel et pour m'avoir aidé avec ses conseils utiles.

Aux merveilleux gens que j'avais rencontrés durant mon séjour à Valenciennes : Ali, Sadok, Frédéric, Ioana, Amina, Souhaila, Safwen, Djamel, Marwen, Eduardo, Blanca, Othmen, Hamada, Amjad, Danila, Aymen, Ziad, Mehdi, Zaki, Imed, Ahmed, Fadi, Mouna, Mariem...

A l'équipe du LAMIH que j'ai eu la chance de connaître : Zeineb, Sawsen, Yunfei, Karim, Ajie, Hanene, Valérie, Issmet, Yahia, Marko...

A des gens que la vie m'a donné l'honneur de rencontrer, qui ont été toujours là malgré la distance et les occupations de la vie : Olfa, Abir, Abderrahim, Haifa, Anissa, Hafedh, Baraket, Youssef, Rany, Malek, Neyrouz, Souha.

Sinon :
Ce n'était, en aucun cas, de simples années passées à l'INSAT. C'était toute une vie, un

voyage à la quête d'une nouvelle personnalité cachée au fond des grottes de mon âme. Un voyage au cours duquel j'ai appris, énormément appris, des leçons pour la vie. Maintenant, Le voyage est terminé mais la mutation n'est jamais accomplie. Il faut juste fixer une nouvelle destination pour qu'elle continue son boulot.

L'INSAT a beaucoup donné pour moi. Je suis énormément reconnaissant à ce lieu, ces briques rouges qui me rappellent une infinité de choses, ces gens aux belles âmes et aux grands cœurs. L'INSAT est une époque que je n'oublierai jamais dans ma vie. Je raconterai un jour à mes fils et à mes petits ce que j'ai vécu au cœur de cet institut.

Durant ces 5 ans, j'ai eu la chance de vivre des moments uniques en leur genre, des instants qui seront gardés dans ma propre boîte à souvenirs. J'ai eu la chance aussi de rencontrer des gens qui méritent bel et bien le titre d'Homme (avec un Grand H). Ce sont des gens sans lesquels l'histoire n'aurait jamais un sens. Et donc, il faut leur rendre hommage. C'est pour cela que je dédie ce travail à :

Les anciens combattants : Maher, Chiheb, Abouda & Lyès (pour tout ce que vous m'avez appris depuis une éternité), Seif (Media Master), Marwen & Imen (for believing in me once upon a time), Aicha (Tata), Chouki (le nom suffit)

Los compañeros del ejército del IIA y del Aerobotix INSAT : Balha (binôme, délégué et un vrai compagnon de route), Majdi (Bagra), Ayadi (moudirna), Charrada (Mr the president), Ilyes & Yassine (les inséparables), Darghouth (DMA, STM in his blood), Meriem (Sagem n'est qu'un souvenir lointain déjà), Rawen (Mes créations), Yasmine Jabeur (le binôme de la révision all seasons), Toumi (weld l'bled), Aziza (le Facteur clé de succès de cette promotion), Slim (ta notion de temps est admirable), Marwen Saafi (ton nom suffit)

La familia Tunirobots : Safa, Rania & Aicha (people of TR), Cyrine, Iheb, Lys (the best media team in the world). Rania Touihri (it was a pleasure to work with you), Mohsen (notre cher amérindien), Guenbri (elmoudir), Mehdi (Challenge man), Rym (Gaddour tout court), Sakr (ce fut un honneur de travailler avec toi, président). Raya (moula el bach), Mariem (ha

madame), Zied (l'électrochoc est une nécessité), Karim, Ayari (it's all about money), Amen (GGMU)

La familia INSAT-press (Wood faces) : Erij (You know & I know), Feriel (La plus talentueuse des pâtissières, Fery Bakery), Haypha (The grammar nazie), Ines (la bachelière et la plus jeune des pressiens), Sirine (bent el7ouma et la chargée de mon transport), Saoussen (ton café est toujours délicieux), Asma (Nain la caricaturiste), Sophia (ta plume est une merveille), Abir (palestinian girl), Ines (la photographe)

Aerobotix & Aeroday people : Chawki (choko), Mehdi (the old school man), Omar (The doctor), Maroua (Mrs the president), Dorra, Msadaa (Mes respects), Rabeb (7med)

La familia CFE : Abir, Sahbi, Walid, Marwa, Meriem, Mahmoud, Emna, Hana (merci pour m'avoir adopté lors de ma première année), Fethi (football is my DNA)

La familia Securinets : Seif (Big Seif), Mouna (Japaneese girl), Taher, Manel (l'atelier, l'atelier), Salah, Empereur (tu seras un grand chanteur dans un monde parallèle), Amjed, Soumaya et Anas (Challenge people), Salma, Wiem (tweety), Yassin (a9wa zamil), Aly (Mr Sponsoring).

Il faut aussi remercier des gens que j'avais la chance de les rencontrer dans ce merveilleux monde : Habib, Omar (Mia San Mia), Erij Temimi (jarty), Manel (la randonneuse), Marwa (such a great friend), Wassim (un grand compagnon de route), Tito (le présent dans les grèves et l'absent ailleurs), Nesryne (un Slim au féminin)

Il y a aussi les champions qui nous ont quittés déjà : Sarhan (the geek), Bedis (l'autre Geek), Tarek (le néerlandais), Aymen (ça fait un bail), Sami (we miss you), Mehdi (meedoch), Zahmoul (je te maudis parfois, mais je ne peux pas te haïr)

Sans aussi oublier la nouvelle génération : Rania (Jazz), Yasmine (insta girl), Alaa (theatro), Ghassen (The gerd), Houcem (eightish one), Teissir (sa7by w sadi9i), Maryem (pure spirit that I have met), Myriam (the coach is really proud of you), Hela (l'autre délégué), Ihssen

(tu feras quelque chose de grandiose dans ta vie), Koubaa (Nao), Marouane, Belhassen (Ze geek), Asma (my Dear Asma). Vous portez maintenant le flambeau. Gardez-le

Finalement, je dédie ce travail à une personne que j'ai eu la chance de la rencontrer pendant une poignée de main de minutes. Lors de cette rencontre, elle m'a dit « Tu souris trop, tu auras un avenir prospère». Madame, ce travail n'est que le début. Je n'oublierai jamais ces mots.

The INSAT chapter is over... A new challenge is going to begin... & As he said the Mohsen « bosser un chinois pour célébrer comme un amérindien »... Il est temps de célébrer le passage vers le pallier suivant

Remerciements

Aux termes de ce travail, je remercie, Mr Slim BEN SAOUD, Professeur à l'Institut National des Sciences Appliquées et de Technologie, pour avoir accepté de présider ce jury.

Je remercie, Mr Mohamed Sahbi BELLAMINE, maître assistant à l'Institut National des Sciences Appliquées et de Technologie, pour avoir examiné ce travail.

Je tiens spécialement à remercier mes encadrants Monsieur Abderrazak JEMAI, Maître de conférences en Informatique et Monsieur Mustapha HAMDI, Maître assistant à l'Institut National des Sciences Appliquées et de Technologie, pour leurs précieuses directives, tout leur temps sacrifié à ma formation et leur confiance.

Je tiens aussi à exprimer mon profond respect et toute ma gratitude à tout le personnel du Laboratoire d'Automatique, de Mécanique et d'Informatique Industrielles et Humaines (LAMIH), notamment au Maître de conférence Rabie BEN ATITALLAH et au Maître de conférence David DUVIVIER pour m'avoir aidé et soutenu à réaliser ce travail

Pour mes enseignants de l'Institut National des Sciences Appliquées et de Technologie, je vous suis très reconnaissant pour la formation de qualité que vous m'avez offerte, pour votre disponibilité, votre soutien et votre présence.

La règle 15 : "Dieu s'occupe d'achever ton travail, intérieurement et extérieurement. Il est entièrement absorbé par toi. Chaque être humain est une œuvre en devenir qui, lentement mais inexorablement, progresse vers la perfection. Chacun de nous est une œuvre d'art incomplète qui s'efforce de s'achever." Shams de Tabriz

La règle 33 : "Tandis que chacun, en ce monde, lutte pour arriver quelque part et devenir quelqu'un, alors que tout cela restera derrière eux quand ils mourront, toi, tu vises l'étape ultime de la vacuité. Vis cette vie comme si elle était aussi légère et vide que le chiffre zéro. Nous ne sommes pas différents de pots : ce ne sont pas les décorations au-dehors, mais la vie à l'intérieur qui nous fait tenir droits." Shams de Tabriz

"Anyone who wants to see the sunlight clearly needs to wipe his eyes first." Saint Athanasius

Table des matières

Liste des figures xi

Liste des tableaux xii

INTRODUCTION GENERALE 1

1 MISE EN CONTEXTE DU PROJET 3

1.1 Introduction 4

1.2 Présentation de l'Unité d'acceuil : 4

1.2.1 Université de Valenciennes et du Hainaut Cambrésis (UVHC) : 4

1.2.2 Laboratoire d'Automatique, de Mécanique et d'Informatique Industrielles et Humaines (LAMIH) : 6

1.2.2.1 Organigramme du LAMIH : 7

1.2.2.2 Effectif du LAMIH : 8

1.2.2.3 Département Informatique du LAMIH 8

1.3 Contexte du projet : 9

1.3.1 La collaboration Airbus-Helicopters-LAMIH : 9

1.3.2 Le projet DTANAV : 10

1.4 Etude de l'existant : 10

1.5 Problématique : 15

1.6 Plan d'action : 15

1.7 Planification du projet : 16

1.8 Conclusion : 16

2 ANALYSE DE L'EXISTANT EN 2D, ETUDE DE COMPLEXITE ET OPTIMISATIONS FAITES 17
2.1 Introduction : . . . 18
2.2 Analyse de l'existant : . . . 18
2.2.1 Présentation de l'outil de planification de vol : . . . 18
2.2.2 La méthode de discrétisation : Les Quadtrees : . . . 18
2.2.2.1 Avantages & Inconvénients des quadtrees : . . . 19
2.2.2.2 Principe de fonctionnement des quadtrees : . . . 21
2.2.3 L'algorithme du chemin le plus court : l'algorithme A* : . . . 23
2.2.3.1 Principe de fonctionnement de l'algorithme A* . . . 26
2.2.3.2 Principe de fonctionnement de l'algorithme Thêta* : . . . 28
2.3 Etude de la complexité : . . . 30
2.3.1 Etude de la complexité temporelle : . . . 32
2.3.2 Etude de la complexité spatiale : . . . 35
2.4 Optimisation niveau temps d'exécution et mémoire : . . . 37
2.5 Conclusion : . . . 39

3 MISE EN PLACE DE L'OUTIL DE PLANIFICATION DE VOL EN 3D 40
3.1 Introduction . . . 41
3.2 Analyse de l'existant : Les Octrees . . . 41
3.2.1 Avantages et Inconvénients des Octrees : . . . 42
3.2.2 Principe de fonctionnement des Octrees : . . . 42
3.2.3 Conclusion : . . . 45
3.3 Solution proposée : . . . 45
3.3.1 Principe de la solution : . . . 45
3.4 Test & Résultats : . . . 48
3.4.1 Résultats sur le terrain de l'Andorre : . . . 48
3.4.2 Résultats sur le terrain des Pyrénées : . . . 49
3.4.3 Résultats sur une falaise : . . . 50
3.5 Etude de la complexité temporelle et spatiale : . . . 52

3.5.1 Etude de la complexité temporelle : 52
3.5.2 Etude de la complexité spatiale : 55
3.6 Conclusion : 56

4 IMPLEMANTATION EMBARQUEE 57
4.1 Introduction 58
4.2 Présentation de l'environnement embarqué : 58
4.2.1 Présentation de la plateforme : 58
4.2.2 Présentation de la partie AP SOC : 61
4.2.2.1 Présentation de la partie Logique programmable : 62
4.2.2.2 Présentation de la partie processeur : 62
4.3 Implémentation embarquée sur la carte Zybo 64
4.3.1 Installation de Xilinux : 65
4.3.2 Paramétrage de la carte : 66
4.3.3 Mise à niveau de la carte : 66
4.3.4 Création de l'environnement de développement : 67
4.3.5 Téléversement du code vers la carte : 67
4.4 Analyse des résultats : 67
4.4.1 Analyse côté temps d'exécution : 67
4.4.2 Analyse côté mémoire : 70
4.5 Conclusion : 71

CONCLUSION GENERALE ET PERSPECTIVES 73

Table des figures

1.1 Logo de l'Université 4
1.2 Logo du LAMIH 6
1.3 Organigramme du LAMIH 7
1.4 Effectif du LAMIH 8
1.5 Liste des chercheurs du LAMIH 8
1.6 Digramme de cas d'utilisation 11
1.7 Diagramme de classe 12
1.8 La représentation 3D d'un fichier asc 13
1.9 Planning prévisionnel du stage 16

2.1 Représentation des Pyrénées à 150m 22
2.2 Représentation des Pyrénées à 1500m 22
2.3 Représentation des Pyrénées à 2000m 22
2.4 Représentation des Pyrénées à 3300m 23
2.5 Les variantes du A* 24
2.6 Pseudocode de l'algorithme A*[11] 27
2.7 Calcul des chemin selon le Thêta*[18] 28
2.8 Pseudocode de l'algorithme Thêta* 29
2.9 Graphe 2D sur le terrain des Pyrénées à 1500m 30
2.10 Processus de l'outil en 2D 31
2.11 Organigramme de l'outil en 2D 31
2.12 Evolution du temps de génération du graphe en fonction de l'altitude et du Radius Factor 35

2.13 La nouvelle architecture du projet 36
2.14 Courbe de l'évolution du temps de génération du graphe dans la version optimisée 38

3.1 Un exemple d'un octree 41
3.2 La discrétisation d'un objet par les octrees[15] 43
3.3 L'arbre généré par la discrétisation de l'objet[15] 43
3.4 L'Andorre représenté par les octrees 44
3.5 La région de Grenoble représentée par les octrees 44
3.6 Les Pyrénées représentés par les octrees 45
3.7 Le processus de l'outil en 3D 46
3.8 Organigramme de l'outil en 3D 47
3.9 Diagramme de classe de la version 3D de l'outil 47
3.10 La vue complète du graphe sur l'Andorre 48
3.11 La vue à côté du graphe 48
3.12 La vue 3D du graphe sur les Pyrénées 49
3.13 La vue à côté du graphe 50
3.14 La vue complète du graphe sur la falaise 51
3.15 La vue du dessus du graphe 51

4.1 La carte Zybo Zynq 7000 58
4.2 Organisation de la plateforme[9] 60
4.3 l'Architecture AP SOC du Zynq[9] 61
4.4 La FPGA ARTY appartenant à la famille Artix-7 62
4.5 L'architecture du Cortex-A9[2] 64
4.6 Etapes de l'implémentation 64
4.7 Logo d'Ubuntu 66
4.8 Evoltuion du temps d'exécution en fonction du Radius Factor 68
4.9 Comparaison entre la FPGA et un ordinateur 70

Liste des tableaux

2.1 Etude Comparative entre les quadtrees et les BSP 20
2.2 Evolution du temps d'exécution des différentes phases de l'algorithme 34
2.3 Quantité de la mémoire utilisée pour différentes configurations 36
2.4 Evolution du temps des différentes phases de l'algorithme dans la version optimisée 37
2.5 Quantité de la mémoire allouée pendant chaque phase de l'algorithme optimisé 38

3.1 Avantages et Inconvénients des Octrees 42
3.2 Analyse du temps d'exécution de l'outil en 3D 54
3.3 Quantité mémoire allouée aux données par l'outil 55

4.1 Les caractéristiques de la carte Zybo[9] 59
4.2 Temps d'exécution en fonction du Radius Factor 68
4.3 Evolution des temps des différentes phases d'exécution 69
4.4 Evolution de la quantité de mémoire allouée aux données 71

INTRODUCTION GENERALE

Planifier une mission de vol pour un hélicoptère s'avère une tâche difficile voire cruciale. La moindre erreur ou la moindre défaillance peut avoir des répercussions graves au niveau matériel et humain. En effet, lors de la planification, il faut prendre en considération plusieurs facteurs : le point de départ et le point d'arrivée, la nature géographique du terrain, le modèle aérodynamique de l'hélicoptère... Ces derniers ont une forte influence sur le plan de vol. C'est pour cela que les plus grandes compagnies aéronautiques accordent une importance majeure à ce sujet. Les département R&D au niveau de ces entreprises cherchent toujours à optimiser et à améliorer les algorithmes de commandes de ces aéronerfs. Tout constructeur avionique cherche à concevoir et à implémenter un algorithme permettant la commande de l'appareil en toute sécurité et l'élaboration du plan de vol le plus sûr, tout en respectant les contraintes d'altitude, d'énergie et du relief.

Dans ce contexte, Airbus Helicopters, un géant de la construction avionique, collabore avec le Laboratoire d'Automatique, de Mécanique et d'Informatique Industrielles et Huaminaes(LAMIH), implanté au sein de l'Université de Valenciennes et du Hainaut Cambrésis (UVHC), pour concevoir et réaliser un système de planification de vol capable de procurer au pilote de l'hélicoptère le chemin optimal entre un point de départ et un point d'arrivée.

Ce système est un outil software d'aide à la décision pour le pilote de l'appareil lui procurant le chemin et la trajectoire à faire. Ensuite, il lui permet d'assurer la replanification de la mission lors de la détection d'un obstacle ou la violation de l'une des contraintes imposées. Le projet, lancé au début de l'année 2010, n'arrête pas d'évoluer. Dans ce contexte, notre mission consiste essentiellement à optimiser un outil de planification de vol qui permet, à

partir d'un fichier donné en entrée, de discrétiser le terrain, trouver un chemin optimal et fournir à l'appareil les points de passage. Cet outil devra ensuite être implémenté sur une carte FPGA pour tester son fonctionnement dans des conditions temps réel.

Le présent rapport est divisé en quatre grands chapitres qui résument l'intégralité du travail réalisé tout au long du stage.
Le premier chapitre de ce rapport sera dédié à la présentation de l'organisme d'accueil du projet ainsi que le contexte et le cadre général. Lors de ce chapitre, on va présenter la problématique à résoudre.
Dans le deuxième chapitre, on procédera à l'amélioration des performances de notre outil software. De ce fait, nous allons présenter l'ensemble des actions d'optimisation faites accompagné d'une étude de complexité de temps et de mémoire.
Le troisième chapitre va expliquer le développement et le déploiement de la version 3D de l'outil dans le but de permettre la navigation dans l'espace. On va aussi faire l'étude de la complexité en termes de mémoire et du temps d'exécution de notre outil de planification dans le même contexte.
Le quatrième chapitre sera consacré à la partie implémentation de notre outil de planification de vol sur une FPGA et à la phase de test sur la plateforme embarquée.

Chapitre 1

MISE EN CONTEXTE DU PROJET

Sommaire

1.1 Introduction 4
1.2 Présentation de l'Unité d'acceuil : 4
1.2.1 Université de Valenciennes et du Hainaut Cambrésis (UVHC) : . . 4
1.2.2 Laboratoire d'Automatique, de Mécanique et d'Informatique Industrielles et Humaines (LAMIH) : 6
1.2.2.1 Organigramme du LAMIH : 7
1.2.2.2 Effectif du LAMIH : 8
1.2.2.3 Département Informatique du LAMIH 8
1.3 Contexte du projet : 9
1.3.1 La collaboration Airbus-Helicopters-LAMIH : 9
1.3.2 Le projet DTANAV : 10
1.4 Etude de l'existant : 10
1.5 Problématique : 15
1.6 Plan d'action : 15
1.7 Planification du projet : 16
1.8 Conclusion : 16

1.1 Introduction

Dans tout projet, il semble nécessaire voire évident de mettre en place le contexte du projet en présentant l'unité d'acceuil ainsi que l'établissement de la problématique autour de laquelle tourne le projet.

Ce chaptire est composé de deux volets. Dans le premier volet, nous allons présenter l'unité d'accueil, son organisation, ses activités et les axes sur lequels elle travaille. Ensuite, le deuxième volet de ce chapitre parviendra à mettre le projet dans son cadre, délimiter son étendu et établir la probématique sur laquelle on va travailler.

1.2 Présentation de l'Unité d'acceuil :

1.2.1 Université de Valenciennes et du Hainaut Cambrésis (UVHC) :

L'Université de Valenciennes et du Hainaut Cambrésis a vu le jour en 1964. Au début, c'était une antenne de la faculté des sciences de Lille. Tout a commencé par un cours de mathématiques. L'embryon universitaire se développe rapidement par des seconds (1971) et troisème cycles (1975), à vocation professionnelle, créés à partir de jeunes laboratoires de recherches en relation avec le monde industriel et les milieux économiques nationaux.
Un réseau de labratoires de recherches et d'ateliers de transferts performants permet à l'UVHC de participer activement à la conversion économique du Hainaut-Cambrésis[5]



Figure 1.1 – Logo de l'Université

Le premier cours de Mathématiques a été donné le 04 Novembre 1964. D'autres étapes importantes ont marqué le développement de notre Université.

Voici les dates clés qui marquent le développement de l'université :

• 1964 : Centre scientifique universitaire, aujourd'hui l'institut des sciences et techniques (ISTV)

• 1967 : Institut universitaire de technologie (IUT)

• 1969 : 1er laboratoire de recherche opto-acousto-électronique, aujourd'hui IEMN-CNRS

• 1970 : Statuts du centre universitaire et Faculté des lettres, langues et arts (FLLASH)

• 1978 : Faculté de droit, d'économie et de gestion (FDEG)

• 1979 : Statuts de l'université et Ecole nationale supérieure d'ingénieurs de mécanique énergétique (ENSIMEV)

• 1982 : Antenne à Cambrai et Institut de préparation à l'administration générale (IPAG)

• 1983 : Antenne à Maubeuge

• 1986 : CRITT à Maubeuge (Centre régional d'innovation et de transfert de technologie céramiques fines)

• 1989 : Création de Valuval : cellule de valorisation de la recherche

• 1991 : Ecole d'ingénieurs en génie informatique et productique (EIGIP), Institut supérieur industriel de Valenciennes (ISIV), en partenariat avec l'union des industries des métiers de la métallurgie (UIMM) , Antenne de l'Institut universitaire de formation des maîtres (IUFM)

• 1995 : Institut d'administration des entreprises (IAE), Faculté des sciences et des métiers du sport (FSMS) et Centre technologique en transports terrestres (C3T)

• 1999 : Valutec SA, filiale de l'université, pour favoriser le transfert de technologie

• 2002 : Ecole nationale supérieure d'ingénieurs en informatique, automatique, mécanique et énergétique (ENSIAME) : fusion de l'ENSIMEV, de l'EIGIP et de l'ISIV.

LMD : l'université est l'une des premières en Europe pour l'harmonisation de ses diplômes dans le cadre du LMD (Licence-Master-Doctorat)

• 2008 : label campus innovant « transport durable »

• 2014 : l'Université fête ses 50 ans [4]

1.2.2 Laboratoire d'Automatique, de Mécanique et d'Informatique Industrielles et Humaines (LAMIH) :

Le LAMIH UMR CNRS 8201 (Laboratoire d'Automatique, de Mécanique et d'Informatique Industrielles et Humaines) est une unité mixte de recherche entre l'Université de Valenciennes et du Hainaut-Cambrésis (UVHC) et le Centre National de la Recherche Scientifique (CNRS). Sa nouvelle structure est issue de la fusion au 1er janvier 2015 avec le laboratoire TEMPO EA 4542 Carnot ARTS. Le LAMIH est organisé en 4 départements disciplinaires bien identifiés : Automatique, Mécanique, Informatique, Science de l'Homme et du Vivant (SHV). Au sein de chaque département existent des thèmes de recherche (2 ou au maximum 3) qui peuvent évoluer au cours du temps et des différentes roadmaps à venir. Il a un rattachement principal à l'INS2I où il relève des sections 06 pour le département Informatique et 07 pour le département Automatique. Il a deux rattachements secondaires, à l'INSIS sections 09 et 10 pour le département Mécanique et à l'INSB section 26 pour SHV. [6]



Figure 1.2 – Logo du LAMIH

Le LAMIH (Laboratoire d'Automatique, de Mécanique et d'Informatique Industrielles Humaines) est issu d'un projet de fusion à partir des équipes du LAMIH et du TEMPO (Thermique, Energétique, Mécanique et Matériaux, Procédés de mise en forme production), sous la forme de quatre départements. Ainsi, les équipes « Automatic, control and human-machine systems » (ASHM) du LAMIH et « Production, service, information » (PSI) de TEMPO donnent naissance au département Automatique ; les équipes « Crash, comfort and safety » (C2S) du LAMIH, « Material, surfaces and forming » (MSM) de TEMPO et « Fluid dynamics

and heat transfer » (DF2T) de TEMPO se regroupent au sein du département Mécanique ; les équipes « Decision, interaction and mobility » (DIM) du LAMIH et « Decision, emotion and human motricity » (DEMoH) du LAMIH deviennent respectivement le département Informatique et le département Sciences de l'homme et du vivant de la nouvelle unité.[19]

1.2.2.1 Organigramme du LAMIH :

La recherche du LAMIH est structurée en quatre départements, chacun desquels sont dirigés par un directeur et un directeur adjoint. Sous le couvert du Bureau de Laboratoire, trois cellules assurent un service commun à l'ensemble des départements :

• Une cellule « Administrative et financière » pour la gestion des ressources humaines, la gestion financière et les achats.

• Une cellule « Evènementiel et communication » pour l'organisation d'évènements tel que les conférences, les congrès, les salons, etc.

• Une cellule « Ressources Transverses » comprenant la gestion des systèmes d'informations, le centre de compétence technique et le service « Hygiène et sécurité ».

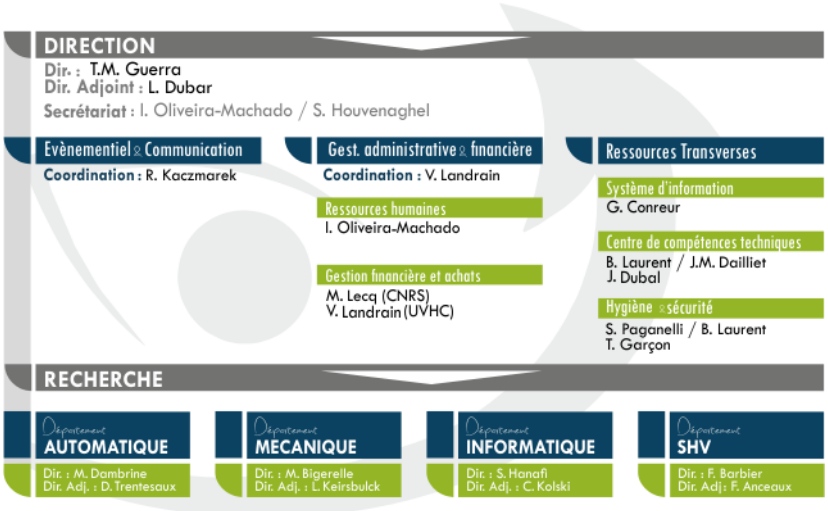


Figure 1.3 – Organigramme du LAMIH

D'un point de vue de l'utilisation des nombreux moyens d'essais, le LAMIH a entamé depuis plusieurs années une politique de qualité et de sécurité notamment par la formation des assistants de prévention et le déploiement de procédures spécifiques. [7]

1.2.2.2 Effectif du LAMIH :

Le LAMIH UMR CNRS 8201 est composé de 4 départements dont les effectifs en termes de permanents sont donnés ci-après. On rappelle pour chacun les sections CNU ministère et les instituts et sections du CNRS concernées.

	Professeur	Maitre de conférences	CNRS*	CNU*	BIATSS ITA	Administratifs	
Automatique	18	17	INS2I	61ème	9	-	
Informatique	12	18	INS2I	27ème	-	-	
Mécanique	17	20	INSIS	60ème,62ème	13	-	
SHV	1	7	INSB	71ème,74ème	2	-	
Transversal						8	
Total	**48**	**62**			**24**	**8**	**142**

Figure 1.4 – Effectif du LAMIH

Le LAMIH dispose également de ressources transversales administratives (voir organigramme) composé de 2 secrétaires de direction, d'une responsable de la gestion administrative et financière, 3 secrétaires en charge des départements et de 2 responsables de gestion financière et achats pour la partie CNRS.
Le tableau suivant résume les effectifs en incluant les non permanents (CDD, doctorants et post-doc)

	CDD	Doctorants	POST-DOC	
Automatique	8	21	2	
Informatique	5	33	2	
Mécanique	4	34	2	
SHV	3	6	1	
Total	**20**	**94**	**7**	**121**

Figure 1.5 – Liste des chercheurs du LAMIH

1.2.2.3 Département Informatique du LAMIH

Le département Informatique du LAMIH UMR CNRS 8201 est composé de près de 70 membres dont 30 chercheurs permanents, 40 doctorants, post doctorants et ingénieurs, et une secrétaire. Les activités de recherche des membres permanents de ce département portent

principalement sur la Décision (modèles, outils et techniques de Recherche Opérationnelle et d'Intelligence Artificielle pour l'aide à la décision), l'Interaction (Modèles d'interaction intégrant l'Humain), et la Mobilité (développement de systèmes complexes/embarqués, intégrant l'Humain, pour l'aide à la décision) essentiellement mais non exclusivement dans les domaines du Transport et de la Logistique. La recherche du département est scindée en deux thèmes : Optimisation et Mobilité (OptiMOB), et Interaction et Agents (InterA).

Au niveau du département, nous étions affectés à l'équipe OptiMOB. Le thème Optimisation et Mobilité (OptiMob) est constitué de chercheurs travaillant dans les domaines de la Recherche Opérationnelle, l'Aide à la Décision, l'optimisation et les Systèmes Mobiles et Embarqués. Les membres de ce thème travaillent sur la conception d'approches hybrides combinant les métaheuristiques, la simulation et les méthodes mathématiques pour l'optimisation combinatoire, globale, non linéaire. Ils contribuent à la modélisation et l'optimisation des systèmes complexes, dynamiques, stochastiques.

1.3 Contexte du projet :

Notre projet vient dans un contexte particulier. Il s'agit d'une collaboration entre le département informatique du LAMIH et Airbus Helicopters pour la réalisation un objectif bien déterminé.

1.3.1 La collaboration Airbus-Helicopters-LAMIH :

Comme toute unité de recherche, le LAMIH a besoin de réaliser des collaborations avec des industriels afin de financer ses activités de recherche et surtout pour donner un aspect concret à ses travaux puisque « la théorie sans la pratique est impuissante » (Emanuel Kant). Cette collaboration, datant de 2010, porte sur le volet planification de vol et recherche du chemin le plus court.

Cette collaboration a monté des thèses, des stages, des contrats de recherche. Elle a donné naissance à des brevets et à des articles scientifiques parus soit dans des journaux, des revus

ou aussi des articles acceptés dans des conférences dans les quatre coins du monde.

Le but de cette collaboration est de permettre à Airbus de surmonter le problème de planification de vol statique. En effet, Airbus avait un problème dans la replanification des chemins de ses appareils. Une fois que le chemin est fixé, on ne peut plus le changer. Il faut donc penser à une planification dynamique en cas de danger. Ceci est fait en présentant une base de données riche en informations pour permettre une visibilité totale sur la situation.

1.3.2 Le projet DTANAV :

La finalité de cette collaboration s'agit de concevoir et de réaliser un outil de planification de vol qui sera introduit dans l'hélicoptère. Lancé à travers une thèse de doctorat, le projet est passé d'un travail théorique à un outil informatique. Cet outil analyse le terrain et procure tous les éléments nécessaires afin de trouver le chemin optimal en fonction de certains critères définis par le client. Cet outil pourra être utilisé en temps réel dans le cockpit de l'appareil lors du vol. Du coup, on peut reconfigurer le plan de vol de l'appareil lorsqu'un problème surgit lors de l'opération de vol.

Nous allons maintenant commencer notre phase d'analyse de l'existant en vue d'obtenir une visibilité totale sur le projet. Ceci nous permettra ensuite de déterminer la problèmatique majeure du projet et d'en déduire le plan d'action à réaliser pendant le stage.

1.4 Etude de l'existant :

Le projet est présent maintenant en une application console développée avec le langage C sous la distribution Debian de Linux. L'application prend en entrée un fichier satellite contenant les coordonnées géographiques du terrain et les altitudes du relief, une altitude bien déterminée et un radius factor. Elle permet à partir de ces données en entrée de discrétiser le terrain, donner les cases vides et occupées lors de la discrétisation ainsi que la création du graphe correspondant et l'établissement du chemin le plus court. Ces fonctionnalités sont ensuite suivies d'un affichage de certain nombre de courbes pour évaluer les performances de notre

outil. Chacune de ces étapes sont accompagnées par une mesure du temps d'exécution pour garder des statistiques.

On peut décrire le fonctionnement de notre outil par le diagramme de cas d'utilisation décrit par la figure ci-dessous :

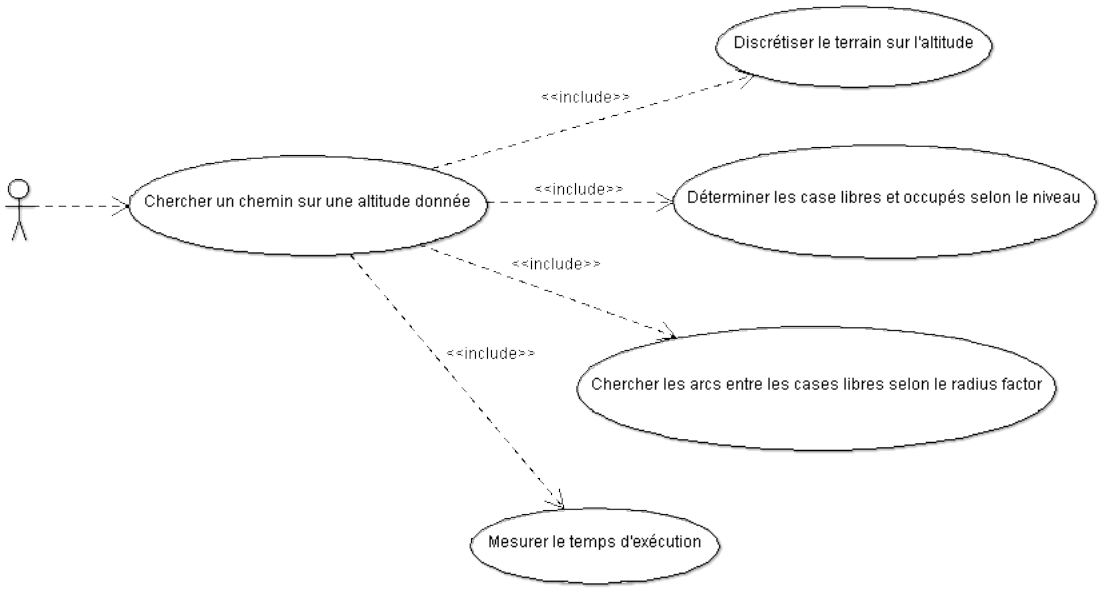


Figure 1.6 – Digramme de cas d'utilisation

On enchaîne après avec le diagramme de classe qui représente les différentes classes intervenant dans l'exécution de notre outil. Ce digramme est donné par la figure suivante :

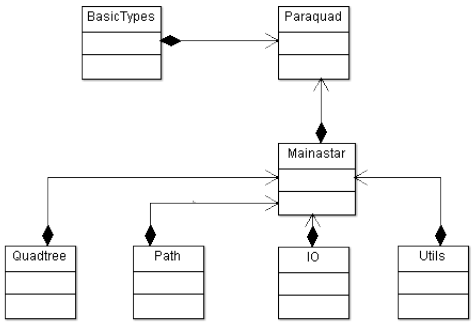


Figure 1.7 – Diagramme de classe

Ce qui suit présentera les définitions de chaque classe de notre outil.

paraquad.c : C'est le fichier main de ce programme. Dans ce fichier, on trouve essentiellement la définition des variables globales ainsi qu'un appel à la fonction « AstarPathPreProcessing ».

mainastar.c : Il contient la fonction « AstarPathPreProcessing ». Cette fonction assure les étapes de lecture du terrain, discrétisation, création du graphe et l'élaboration du chemin. Ces étapes sont séparées par un calcul de la mémoire et du temps à la fin de chacune et elles ne sont réalisées qu'après l'appel aux fichiers cités ci-dessous.

IO.c : C'est un fichier qui contient les fonctions responsables des opérations d'entrée/sortie. Grâce à ces fonctions, on assure la lecture du terrain, l'affichage du graphe et l'écriture dans les fichiers nécessaires.

Quadtree.c : Ce fichier est responsable de la partie géométrie et discrétisation. On y trouve une implémentation des fonctions géométriques ainsi que la discrétisation du terrain en des rectangles inéquitables grâce à la méthode des quadtrees

Path.c : Ce fichier joue le rôle de créateur de chemins. A travers ce fichier, on peut calculer tous les chemins possibles entre les nœuds ouverts obtenus lors de la discrétisation du fichier.

Utils.c : Ce fichier est responsable de la mesure du temps d'exécution de notre outil à la fin de chaque phase.

Il existe une version graphique de notre outil. C'est le produit final que l'industriel demande. En plus des fonctionnalités présentées ci-dessus, cette version permet d'introduire la composante graphique. Il nous permet de visualiser une vue de dessus du terrain traversé ainsi que le chemin à parcourir pendant le vol.

Nos outils prennent en entrée trois paramètres

- La carte : Elle est sous la forme d'un fichier dont l'extension est asc. Le fichier contient les paramètres de la carte : le nombre de lignes et de colonnes de la matrice d'altitude, les positions géographiques (longitude, latitude) et la matrice contenant les altitudes de chaque case.

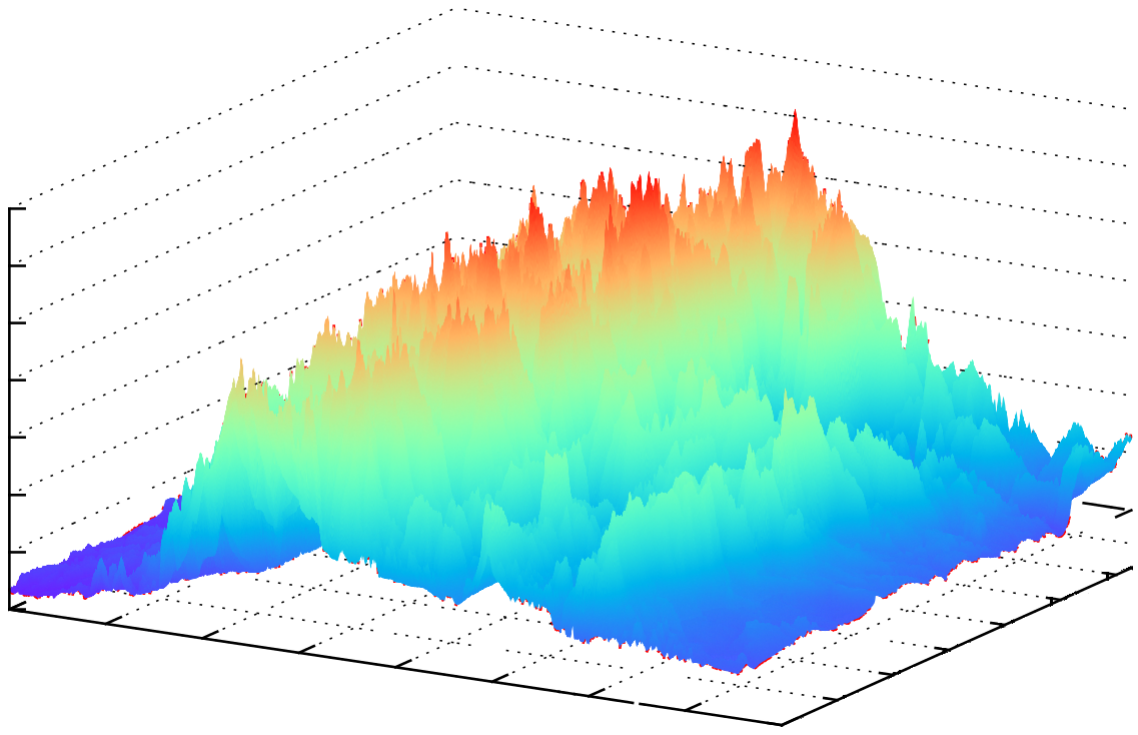


Figure 1.8 – La représentation 3D d'un fichier asc

- L'altitude : Elle est correspondante au niveau sur lequel on va opérer la discrétisation du terrain pour obtenir les cases voulues.

Le Radius Factor : C'est le facteur multiplicatif du rayon terrestre. Il est obtenu par l'équation $k = \frac{1}{1+a(\frac{dn}{dh})}$ où $\frac{dn}{dh}$ est le gradient vertical de l'indice de réfraction et "a" représente le rayon réel de la Terre.[8]

Durant notre stage, nous allons nous focaliser seulement sur l'outil console. Après une phase de test et d'analyse du code source, on a retrouvé plusieurs défaillances et fonctionnalités manquantes :

- Le problème mono niveau : En effet, l'outil prend en entrée une altitude un radius factor bien défini et fait la discrétisation sur ce niveau d'hauteur et avec cette valeur de radius factor. Et donc, le plan de vol calculé ne sera calculé en prenant en considération la montée et la descente de l'appareil. La solution sera donc de passer à une discrétisation multi-niveau, c'est-à-dire, faire la discrétisation en parcourant l'altitude de l'altitude la plus basse à la celle la plus haute avec un pas bien définie. Cela nous permet de prendre en considération tous les niveaux du terrain.

- Le problème de la mémoire : Lorsqu'on a testé les deux outils, on a trouvé une fuite mémoire au moment de la discrétisation du terrain et de la création du graphe. Pour certaines configurations, l'outil se bloque et ne parvient pas à établir un chemin. Ce problème est d'une criticité importante puisqu'il impacte directement la mission de l'aéronef. Il est donc nécessaire voire vital de régler ce problème pour éviter de graves conséquences dans le futur. D'autre part, on a remarqué que nos outils consomment une grande quantité de mémoire. Nos statistiques mémoires montent en exponentielle lorsqu'il s'agit de cartes géographiques de grande taille. Il faut donc agir sur le code source de nos outils de telle sorte qu'on réduise la taille mémoire lors de l'exécution.

- Le problème temps d'exécution : Ce problème est étroitement lié au problème de la mémoire. En effet, lorsque la taille mémoire augmente, le temps d'accès à la mémoire va augmenter. Et

donc, il faut agir sur la mémoire pour pouvoir optimiser le temps. Néanmoins, ceci est vrai à un certain degré. En effet, le paramètre du radius factor est un paramètre impliqué dans le calcul du temps. Lorsque le radius factor est grand, la recherche des voisins devient étalée sur un périmètre plus grand ce qui va augmenter le temps d'exécution.

1.5 Problématique :

Après une analyse de l'existant, nous avons constaté que notre problématique tourne autour de trois idées majeures. Ces dernières constituent les trois principaux axes de notre stage de fin d'études

- Assurer, en 2D, l'exécution de notre outil en minimisant les performances du temps d'exécution et la mémoire.

- Permettre la navigation de l'appareil dans l'espace en passant de la version 2D vers la version 3D

- Implémenter l'outil sur une carte embarquée de type FPGA ayant un design optimal et spécifique pour notre utilisation.

1.6 Plan d'action :

Pour répondre aux trois points cités dans la problématique du projet, nous avons adopté un plan d'action qui contient trois missions à faire tout au long du stage :

- L'amélioration de la solution existante de l'outil en 2D en optimisant la mémoire et le temps d'utilisation. Dans cette partie, on doit aussi étudier la complexité de l'algorithme en terme de temps et de mémoire avant et après les actions d'optimisation pour voir les effets de notre intervention.

- Le passage de la version 2D vers la version 3D grâce à la technique de la discrétisation multi niveaux afin de dépasser les inconvénients des méthodes existantes. Le développement de la version 3D assurera la navigation dans l'espace grâce à un ensemble de connexions reliant les zones libres du terrain.

- L'implémentation de notre outil sur un équipement embarqué pour tester les contraintes temps réel et améliorer le comportement de notre outil vis-à-vis de ces contraintes. L'équipement sera une carte de FPGA dont les caractéristiques ressemblait au calculateur de l'hélicoptère.

1.7 Planification du projet :

Afin d'aboutir à nos objectifs, nous avons mis en place un planning prévisionnel qui définit le temps alloué à chacune des phases de notre projet. Ce planning peut être traduit par la figure ci-dessous :

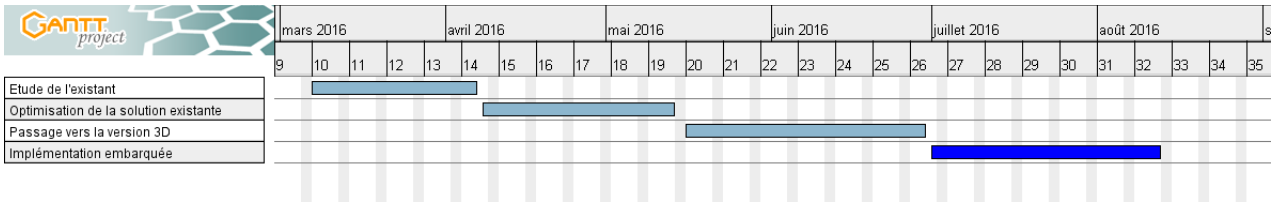


Figure 1.9 – Planning prévisionnel du stage

1.8 Conclusion :

A la fin de ce chapitre, nous avons délimité l'étendu du projet. Nous avons exposé la problématique autour de laquelle tourne le projet à savoir l'explosion temps et mémoire et l'exécution mono niveau. Nous avons proposé un plan d'action qu'on va suivre tout au long du stage. Ce plan d'action va nous permettre d'améliorer l'existant, développer la navigation dans l'espace et tester dans les conditions temps réel. Dans le prochain chapitre, nous allons exposer le travail réalisé pour améliorer la version existante de notre outil et l'étude de la complexité en terme de mémoire et de temps.

Chapitre 2

ANALYSE DE L'EXISTANT EN 2D, ETUDE DE COMPLEXITE ET OPTIMISATIONS FAITES

Sommaire

2.1 Introduction : 18
2.2 Analyse de l'existant : 18
2.2.1 Présentation de l'outil de planification de vol : 18
2.2.2 La méthode de discrétisation : Les Quadtrees : 18
2.2.2.1 Avantages & Inconvénients des quadtrees : 19
2.2.2.2 Principe de fonctionnement des quadtrees : 21
2.2.3 L'algorithme du chemin le plus court : l'algorithme A* : 23
2.2.3.1 Principe de fonctionnement de l'algorithme A* 26
2.2.3.2 Principe de fonctionnement de l'algorithme Thêta* : . . . 28
2.3 Etude de la complexité : 30
2.3.1 Etude de la complexité temporelle : 32
2.3.2 Etude de la complexité spatiale : 35
2.4 Optimisation niveau temps d'exécution et mémoire : 37
2.5 Conclusion : 39

2.1 Introduction :

Pour parcourir un terrain et y trouver le chemin optimal entre deux points, on doit tout d'abord connaître la nature du terrain, le relief, l'altitude et puis établir la route à suivre. Ceci est l'objectif de notre travail et c'est ce qu'on va présenter dans ce chapitre. Le chapitre présent comporte trois parties : Dans la première, on va parler des améliorations apportées au niveau du code source pour améliorer le fonctionnement et obtenir les fonctionnalités désirées. Dans la deuxième partie, on va établir une étude de complexité temporelle et spatiale de notre outil. Ceci nous permettra de cibler les différentes opérations et interventions pour pouvoir optimiser la complexité en temps et en mémoire de notre outil.

2.2 Analyse de l'existant :

2.2.1 Présentation de l'outil de planification de vol :

Comme il a été mentionné dans le chapitre précédent, le projet est un outil développé en langage C sous le système d'exploitation Linux. Cet outil nous permet à partir d'un fichier contenant les altitudes du relief sur une zone bien précise de faire une représentation du terrain sur une altitude passée en entrée. Dans un deuxième volet, notre outil permet d'établir un graph assurant le déplacement dans cette zone représentée.

2.2.2 La méthode de discrétisation : Les Quadtrees :

On définit les quadtrees (arbre quaternaire) comme étant une structure de données dont le type est un arbre. Dans ce dernier, chaque nœud a quatre fils. Les quadtrees sont généralement utilisés pour diviser une espace bidimensionnel d'une matière itérative en faisant une subdivision en quatre nœuds lors d'une itération. Les quadtrees appartiennent à la famille des méthodes kd-trees qui sont un ensemble de méthodes de discrétisations itératives en k fils.

Il existe différentes variantes des quadtrees.

- Le quadtree région : Il permet de représenter une partition de la région donnée en entrée en la subdivisant en quatre quadrants égaux. Ensuite, on répète la même opération jusqu'à ce qu'on obtienne un nœud terminal avec des informations spécifiques à chaque sous-région qu'il représente.

- Le quadtree point : Ce type de quadtree n'est autre qu'une adaptation d'un arbre binaire de recherche usuel. Le quadtree point permet la représentation d'un point à deux dimensions.

- Le quadtree bord : Il est utilisé généralement pour stocker des lignes, contrairement au quadtree point. La discrétisation se fait en approximant les courbes par la division des cellules jusqu'à obtenir une résolution fine. Ceci peut donner naissance à des arbres déséquilibrés.

- Le quadtree carte polygonale : Il est utilisé pour stocker des collections de polygones potentiellement dégénérés. [17]

Dans notre cas, on va utiliser le premier type, à savoir le quadtree région, puisqu'il s'agit de partitionner un terrain comme un espace 2D, avec l'altitude de la zone comme information spécifique.

2.2.2.1 Avantages & Inconvénients des quadtrees :

Par rapport à d'autres méthodes, essentiellement la méthode des BSP, acronyme de Binary Space Partitioning, les quadtrees présentent des avantages et des inconvénients. Mais présentons tout d'abord la méthode des BSP.

« Le partitionnement binaire de l'espace (BSP) est une technique récursive de partitionnement qui a été largement utilisée dans l'infographie depuis le début des années 80[16]. L'algorithme du BSP pour la représentation d'une image [13] prend en entrée une région non partitionnée R (l'image entière initialement), et une ligne l qui a été sélectionnée selon un ensemble de critères. Ensuite, on partitionne R avec l en deux demi-régions R- et R+. Les deux demi-régions peuvent ensuite être partitionnées d'une manière récursive jusqu'au critère d'arrêt. Ceci en résulte une hiérarchie de régions convexes appelées cellules. Une bonne segmentation est obtenue lorsque les valeurs des pixels dans chaque cellule sont homogènes.

Les nœuds d'un arbre BSP, qui ne sont pas des feuilles, sont associés avec les lignes de partitionnement. Les feuilles représentent les cellules de l'image et chaque nœud dans l'arbre représente une région convexe de l'image» [14]
Après une analyse approfondie des deux méthodes, nous pouvons résumer les avantages et les inconvénients des quadtrees par rapport à la méthode des BSP dans le tableau suivant :

	Avantages	Inconvénients
Quadtree	Les Quadtree permettent de donner une représentation détaillée du terrain	Les Quadtree nécessitent plus de bits que les BSP pour le codage
	Les Quadtree est une méthode simple de représentation et de calcul avec des lignes horizontales et verticales	Les Quadtree donnent naissance à une structure complexe d'arbre
	Une Segmentation pertinente de l'image originale	Le nombre de cases générés est volumineux. Ça peut causer un débordement de mémoire
	Un arbre résultant riche en informations	
BSP	Une structure de données simple à manipuler	L'image résultante est pauvre. Elle n'est pas intéressante à explorer
	Le taux d'occupation de la mémoire est faible par rapport aux quadtree	Le codage résultant n'est pas significatif
	L'arbre généré est facile à explorer	Les lignes délimitant l'objet ne sont pas favorables pour le codage
		La technique de segmentation peut passer des objets sans les détecter
		La génération des arbres du BSP est un processus complexe.

Table 2.1 – Etude Comparative entre les quadtrees et les BSP

D'après ce tableau, on peut constater que la comparaison est au profit des quadtrees même si ça va nous causer une explosion en termes de mémoire et de temps d'exécution lorsqu'il s'agit d'un gros terrain. Et donc, on choisit les quadtrees avec la condition d'effectuer des optimisations sur le temps d'exécution et la mémoire.

2.2.2.2 Principe de fonctionnement des quadtrees :

« La décomposition en utilisant la méthode des quadtrees est une simple technique de représentation d'images avec de multiples résolutions. Avec cette technique, l'image est divisée en quatre régions carrées en fonction des activités dans les blocs [25]. La segmentation d'une image avec les quadtrees donne naissance à un arbre dont la racine est l'image originale à la résolution initiale et les quatre carrés égaux sont les fils avec la résolution « niveau 1 ». Chaque pixel dans chaque niveau de résolution a sa propre intensité et l'intensité du nœud parent est la valeur moyenne des intensités de ses fils. Une décision devrait être prise au niveau de chaque nœud pour voir si on procède à la décomposition en quatre cadrants égaux ou on arrête la décomposition. Dans le but d'arriver à une décision, plusieurs mesures d'activités ont été mises en place dans la littérature. [12] Dans chaque nœud, la valeur de la différence absolue est comparée avec une valeur seuil et si la différence est inférieure au seuil, la division est stoppée. Sinon, le nœud est divisé en quatre carrés de tailles égales. Les quadtrees sont les structures de données les plus utilisées pour les applications de traitement d'images. » [14]

Dans le cadre de nos travaux, nous allons appliquer la méthode des quadtrees (arbres quaternaires) sur le cas d'un espace en 2D. Et donc, la discrétisation avec les quadtrees opère avec le principe suivant : On fixe le terrain par le bias du fichier satellite donné en entrée et l'altitude sur laquelle on veut travailler. On lit à partir du fichier une matrice à deux dimensions de tailles variables contenant les altitudes du terrain. Ensuite, on parcourt notre matrice case par case. Chaque fois qu'on trouve un obstacle, c'est-à-dire une case dont l'altitude est supérieure à l'altitude en entrée, on divise cette zone en quatre carrés égaux sinon, on laisse la zone intacte. Cette opération se répète jusqu'à ce que tout le terrain, ou la matrice, soit parcouru. Au final, nous obtenons trois listes, une première contenant les indices des cases libres, une deuxième contenant les indices des cases occupées et une dernière contenant les coordonnées de chaque case. La dernière liste contient les coordonnées du centre de la case et un paramètre indiquant si elle est un obstacle ou non. La première et la deuxième pointent sur la dernière à travers les contenus de leurs cases.
Les figures ci-dessous montrent des exécutions de notre outil sur une carte représentant la

région des Pyrénées pour les altitudes 150, 1500, 2000 et 3300 m.

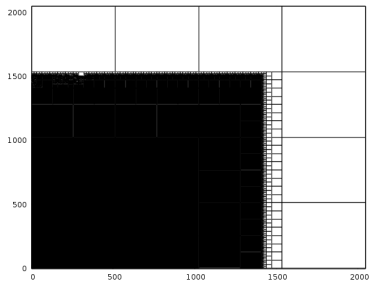


Figure 2.1 – Représentation des Pyrénées à 150m

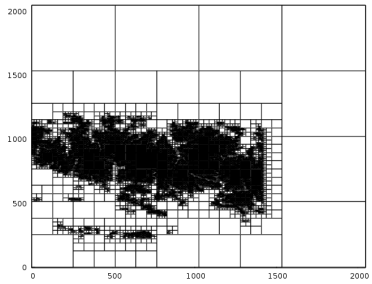


Figure 2.2 – Représentation des Pyrénées à 1500m

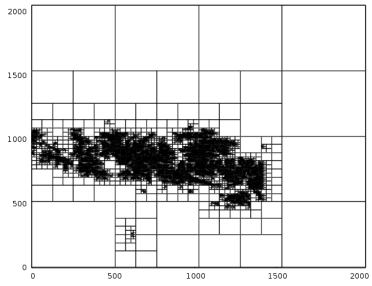


Figure 2.3 – Représentation des Pyrénées à 2000m

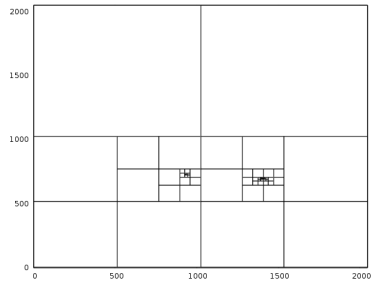


Figure 2.4 – Représentation des Pyrénées à 3300m

On remarque que dans chaque figure :

- Les cases noires correspondent à des obstacles tandis que les cases blanches correspondent à des cases libres où l'appareil peut potentiellement naviguer sans aucun problème.

- Plus on monte dans l'altitude, plus le nombre de cases diminue. Ceci est dû au fait que plus l'altitude est petite, plus on est en pleine zone riche en relief. Donc la discrétisation devient une nécessité ce qui fait bonder le nombre de cases.

- Lorsque la différence entre les altitudes est trop petite, dans notre cas entre 1500 et 2000 m, on ne voit pas de différence entre les images résultantes.

- Explorer le terrain à basse altitude et à haute altitude n'est pas une solution optimale car on fait face à deux cas extrêmes. A basse altitude, on est devant de gros obstacles. Donc, on ne peut pas prédire à quoi ressemble notre chemin car les possibilités de navigation deviennent très limitées. D'autre part à haute altitude, la phase de discrétisation va nous donner de grosses altitudes. Dans ce cas, on n'aura aucune information sur la nature du terrain. Le plan de vol ressemblera probablement à un vol d'oiseau au-dessus du terrain.

2.2.3 L'algorithme du chemin le plus court : l'algorithme A* :

L'algorithme A* est conçu pour permettre la recherche du chemin le plus court dans un graph entre un nœud initial et un nœud passé en paramètres. Le principe de cet algorithme consiste à utiliser une évaluation heuristique au niveau d'un nœud pour explorer les nœuds adjacents

et en choisir le meilleur nœud à visiter. Ensuite, l'algorithme permet la visite de ces nœuds choisis en respectant l'ordre de cette évaluation heuristique.

L'algorithme A* est une variante de l'algorithme de Dijkstra qui a la particularité de réduire le coût d'exécution. En effet, l'algorithme A* est construit sur l'algorithme de Dijkstra associé à une fonction qui estime la distance entre un nœud quelconque du graphe et le nœud cible (par distance, on entend bien sûr le coût du chemin optimal reliant le nœud concerné et le nœud cible). L'idée fondamentale est de modifier le classement dans la liste Ouverts afin de favoriser les nœuds les plus proches du but (au sens de l'estimation) [3]
Au cours du temps, l'algorithme A* a évolué et ait été remis en cause plusieurs fois, ce qui a donné naissance aux variantes représentées par la figure ci-dessous.

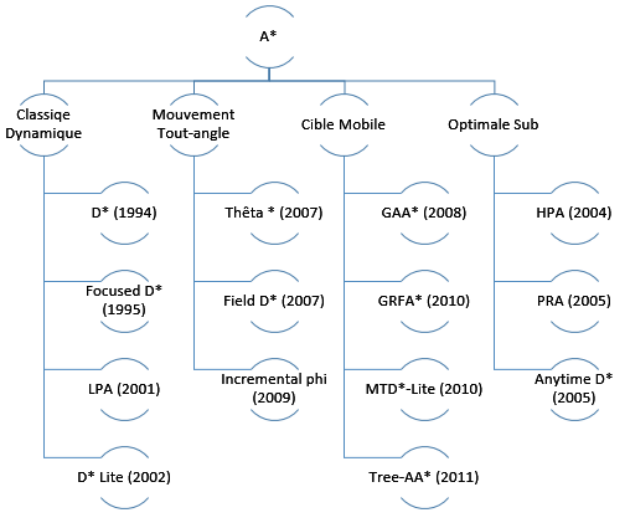


Figure 2.5 – Les variantes du A*

De toutes les catégories énoncées dans la figure, on va se focaliser essentiellement sur les catégories « Les variantes Dynamiques » et la catégorie « mouvement tout-angle » puisqu'elles sont les deux catégories les plus utilisées dans les travaux de planification de chemin.

Les variantes dynamiques de l'algorithme A* sont des algorithmes qui varient selon la dynamique de la navigation. Cette variation est généralement due à des contraintes imposées dans la phase de prétraitement commet des contraintes d'énergie, limitations angulaires, contraintes temporelles... Sous cette catégorie, se trouvent quatre algorithmes :

L'algorithme D* : L'algorithme D* a été introduit en 1994 par Anthony Stentz, chercheur à Carnergie Mellon University. Similaire à A *, D * maintient une liste ouverte des Etats. La liste OPEN utilise des informations sur les modifications pour propager la fonction de coût de l'arc et pour calculer les coûts de chemin aux Etats dans l'espace. Chaque Etat X a un tapis associé (As.uc h que f (X) = NOUVEAU si X n'a jamais été sur OPEN Lisl r (X) = OUVERT si X est actuellement sur la liste ouverte, et r (X) = FERMÉ si X est plus sur la liste ouverte.) [23] La particularité du D* est la possibilité de réagir en temps lorsqu'un changement provient sur le terrain. D'autre part, D* compte pas mal d'inconvénients comme le grand taux de mémoire utilisée et la difficulté d'implémentation.

L'algorithme Focused D* : Cet algorithme est une extension de l'algorithme D*. Il a été créé en 1995 par le même Anthony Stentz. La version focused D * surpasse la version de base, et il offre à l'utilisateur la possibilité de distribuer le calcul la charge entre les parties au bilan et hors- ligne de l'opération, en fonction des exigences de la tâche. L'ajout d'une fonction heuristique de focalisation à D * complète son développement en tant qu'une généralisation de A * à des environnements dynamiques [22]

LPA* : Le LPA* (acronyme de Lifelong Planning A*) est en effet la version itérative de l'algorithme A*. L'algorithme est une combinaison de l'algorithme A* et de l'algorithme itérative Dynamic SWSF-FP (ou l'algorithme SWSF dynamique appliquée au problème du point fixe). Cette combinaison vient du fait que le comportement de l'algorithme SWSF-FP a été modifié pour lui permettre de rechercher le chemin le plus court entre un point et un point d'arrivée et ne s'arrêter que s'il s'assure que c'est le chemin le plus court, ce qui donne le caractère itératif de l'algorithme. D'autre part, LPA* profite du fait que l'algorithme A* utilise les heuristiques pour évaluer le chemin entre deux points. Par conséquent, LPA * combine les avantages de Dynamic SWSF-FP (recherche incrémentale) et A * (recherche heuristique) et est donc potentiellement plus efficace que deux d'entre eux individuellement [20]

D* Lite : Le D* Lite est un algorithme qui permet la replanification rapide du chemin sur un terrain inconnu en utilisant les mêmes stratégies de navigation que l'algorithme D*.

Néanmoins les deux algorithmes ne sont pas implémentés de la même façon. En effet, D* construit sur LPA*, qui a une fondation théorique solide, une grande similarité à A*. [24]. En plus, D* Lite est caractérisé par sa rapidité par rapport à l'algorithme D*.

La catégorie « mouvement tout angle » est la catégorie qui ne prend pas en considération les limitations angulaires. Et donc, les variations de la dynamique seront dues à d'autres contraintes (heuristique, temps, énergie...). Sous cette catégorie, se trouvent trois algorithmes :

Field D* : Cet algorithme est une variante de l'algorithme D* Lite, permettant la planification et la replanification des chemins, dont la particularité est de ne pas aucune limitations angulaire. L'appareil programme peut tourner selon n'importe quel angle. Généralement, même dans les pires scénarios soigneusement construits, le chemin généré par l'algorithme D* est très proche d'être la solution optimale. [12]

Thêta * : Thêta * est une variante de l'algorithme A*. Il combine les idées du A* sur la visibilité du graphe ... et le A* sur les grilles.[18] Et donc, il permet de donner des chemins plus courts que ceux donnés par le A*. Mais, cette variante a un handicap flagrant par rapport à la version de base en termes de rapidité à cause de la définition des nœuds père et nœuds fils.

Incremental phi * : Cet algorithme est la variante incrémentale de l'algorithme Thêta *. Il permet de recalculer rapidement les chemins tout-angles que la réexécution du Thêta * [10] ou encore le phi *, qui est l'extension du Thêta * vers le mode incrémental.

De tous ces algorithmes, nous allons se concentrer sur la version de base, le A* ainsi que le thêta * parce qu'ils sont les algorithmes les mieux adaptés à notre contexte.

2.2.3.1 Principe de fonctionnement de l'algorithme A*

Pour présenter l'algorithme A*, on va tout d'abord définir les notions suivantes :
- g : le coût du chemin parcouru depuis la case de départ jusqu'à la case actuelle

- h : le coût estimé du chemin entre la case actuelle et la case d'arrivée.
- d : le coût du chemin total. Soit pour une case v, d(v) = g(v) + h(v)
- liste des ouverts : la liste des cases libres dans le terrain discrétisé
- liste des fermés : la liste des cases occupées dans le terrain discrétisé

A partir de la case de départ, on commence à évaluer les cases voisines en calculant pour chaque case voisine le coût d. A la base de ces calculs, on choisit la case qui donne le coût le plus minimal possible. Lorsque le choix de la case est effectuée, l'algorithme change sa nouvelle « case de départ » pour celle choisie et affecte l'ancienne dans la liste des fermées. L'algorithme continue son fonctionnement de la même manière jusqu'à atteindre la case d'arrivée.

Néanmoins, cet algorithme présente un point critique qui peut influencer son fonctionnement. Il s'agit de l'évaluation heuristique. En effet, la recherche du plus court chemin dépend fortement de l'heuristique implémentée. Si l'heuristique surestime le chemin prévu, elle peut conduire à des faux choix des cases ce qui peut fausser le chemin produit.
Ce fonctionnement peut être traduit par le pseudocode présenté dans la figure ci-dessous :

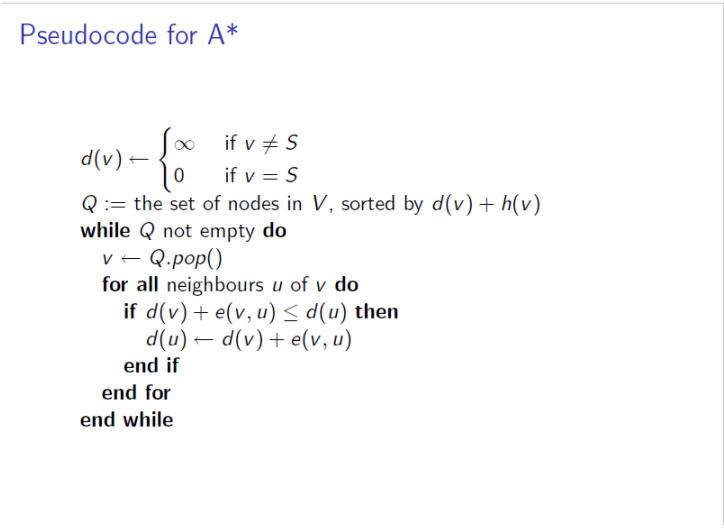


Figure 2.6 – Pseudocode de l'algorithme A*[11]

Où :
- d(v) : la distance entre la case de départ et la case v

- d(u) : la distance entre la case de départ et la case u
- e(v,u) : la distance entre u et v
- Q : La liste des nœuds.

2.2.3.2 Principe de fonctionnement de l'algorithme Thêta* :

Comme il a été introduit précédemment, l'algorithme Thêta * est une variante de l'algorithme de base, le A*, qui ne prend pas en considération les limitations angulaires, vu que c'est un algorithme appartenant à la catégorie « mouvement tout-angle ». Et donc, le principe de fonctionnement de cet algorithme prend celui de l'algorithme A* et y propose en plus une nouvelle méthode de calcul.

En effet, le Thêta * utilise la même méthode de calcul que celle du A*. La seule différence réside dans le calcul du coût du chemin parcouru g. Tandis que le A* se contente d'un seul chemin, le thêta prend en considération deux chemins possibles du nœud de départ vers le nœud actuel :

- Le premier chemin prend en compte le coût entre le nœud de départ et le nœud actuel. C'est le même chemin que propose le A*
- Le deuxième chemin prend en compte le coût entre le nœud de départ et le nœud parent du nœud actuel. Ensuite, l'algorithme essayera de relier le nœud cible avec son parent par un chemin direct.

La figure ci-dessous montre l'algorithme Thêta * appliqué sur un exemple de quadtree.

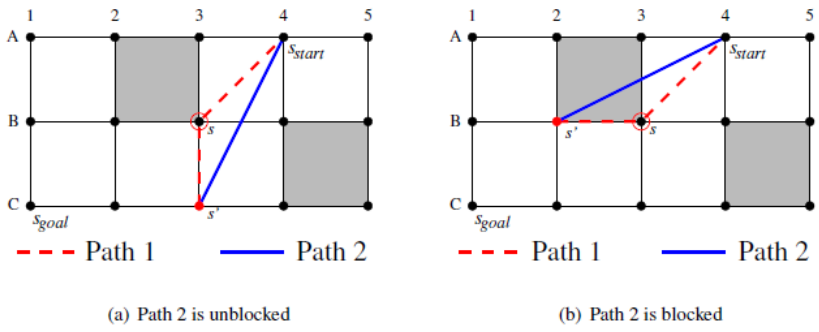


Figure 2.7 – Calcul des chemin selon le Thêta*[18]

Le chemin en rouge présente le chemin calculé en commun avec le A* alors que le chemin en bleu représente le calcul spécifique du Thêta *

Le chemin calculé par le Thêta * (en bleu) est plus court que celui calculé par le A* (en rouge). Ce constat peut être prouvé par l'inégalité des triangles. Cette inégalité dit que la longueur d'un côté du triangle est inférieure à la somme des longueurs des deux côtés restants. Dans tous les cas, le Thêta * choisira le chemin le plus court entre les chemins possibles car l'algorithme ne peut pas passer à travers un nœud occupé par un obstacle comme le montre le deuxième cas de figure.
La figure ci-dessous représente un pseudocode du l'algorithme Thêta * :

```
41 UpdateVertex(s,s')
42     if LineOfSight(parent(s), s') then
43         /* Path 2 */
44         if g(parent(s)) + c(parent(s), s') < g(s') then
45             g(s') := g(parent(s)) + c(parent(s), s');
46             parent(s') := parent(s);
47             if s' ∈ open then
48                 open.Remove(s');
49             open.Insert(s', g(s') + h(s'));
50     else
51         /* Path 1 */
52         if g(s) + c(s, s') < g(s') then
53             g(s') := g(s) + c(s, s');
54             parent(s') := s;
55             if s' ∈ open then
56                 open.Remove(s');
57             open.Insert(s', g(s') + h(s'));
58 end
```

Figure 2.8 – Pseudocode de l'algorithme Thêta*

Dans le cadre de nos travaux, nous avons besoin de générer le graphe complet sur le terrain après la phase de discrétisation pour déterminer la carte de déplacements possibles sur le plan. La figure ci-dessous représente le graphe complet après la discrétisation du terrain des Pyrénées sur une altidude de 1500 m avec un Radius Factor de 1.75.

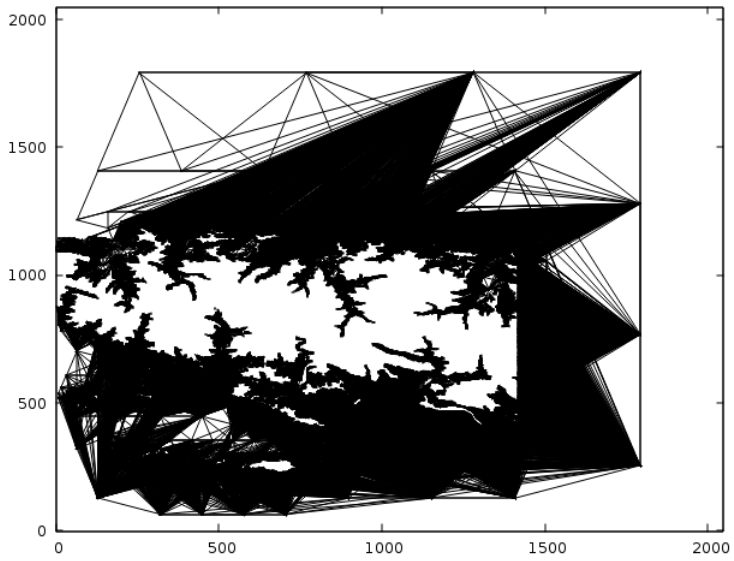


Figure 2.9 – Graphe 2D sur le terrain des Pyrénées à 1500m

Ensuite, à la base de ce graphe et d'une liste des nœuds ouverts sur le terrain résultant, nous allons appliquer l'algorithme A* qui va nous permettre, à partir d'un nœud de départ et un nœud cible choisis aléatoirement, de calculer le chemin optimal à effectuer dans cette zone.

2.3 Etude de la complexité :

Après avoir étudié le fonctionnement de notre outil, on passe maintenant à l'étude de la complexité de notre outil. Cette étude donnera des indicateurs de performances sur notre outil pour ensuite détecter les actions à faire. Cette étude sera sur deux volets :

- La complexité temporelle : L'étude de la complexité temporelle s'intéresse essentiellement à l'étude des temps d'exécutions de chaque phase du travail

- La complexité spatiale : La complexité spatiale concerne l'étude de la mémoire allouée à l'exécution du programme.

Tout d'abord, nous avons besoin de comprendre l'enchaînement que l'outil suit. Il faut mettre tout d'abord en œuvre le processus de notre algorithme. Ce processus contient les étapes à effectuer lors de chaque exécution de l'algorithme. Le processus est décrit par la figure suivante.

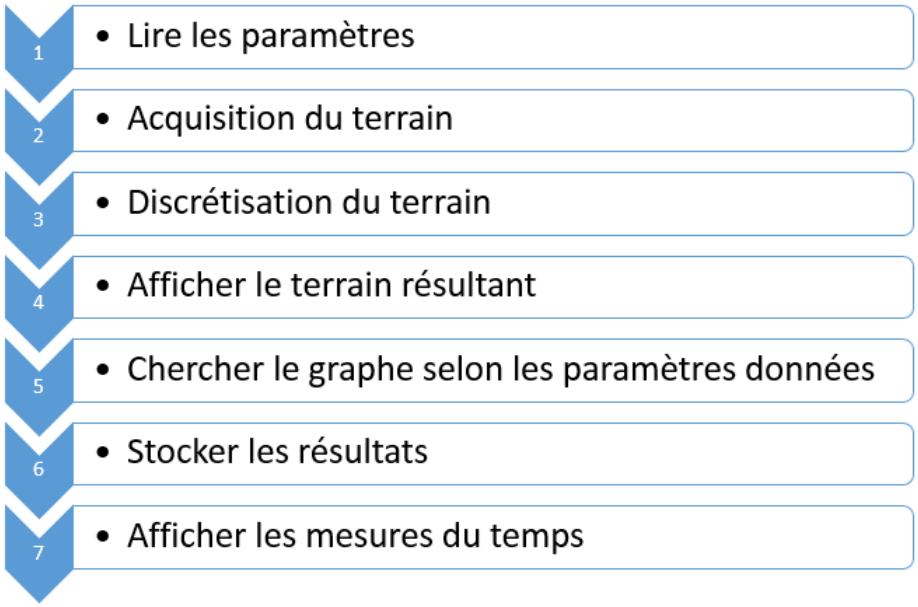


Figure 2.10 – Processus de l'outil en 2D

Ceci peut être traduit par l'organigramme suivant :

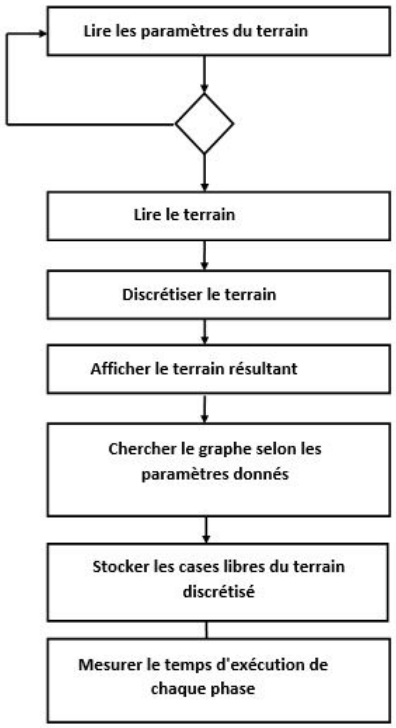


Figure 2.11 – Organigramme de l'outil en 2D

2.3.1 Etude de la complexité temporelle :

L'étude de la complexité algorithme sera une étude « top down », c'est-à-dire qu'on va commencer par le niveau le plus haut pour ensuite étaler l'étude sur les niveaux les plus bas.

En se basant sur notre diagramme de classes établi dans le chapitre précédent, nous commençons toute notre étude sur le fichier paraquad contenant la fonction main. Ensuite, l'étude va entrer dans les détails de l'algorithme.

- Dans le fichier paraquad, la complexité est de O(1) à laquelle on ajoute la complexité de la fonction AstarPathPreprocessing puisqu'il s'agit d'opérations élémentaires. Et donc, on va s'intéresser à la complexité de cette fonction qui nous donnera la complexité globale de notre outil.

- Pour étudier la fonction AstarPathPreprocessing, il faut accéder au fichier mainastar qui contient l'implémentation de cette fonction. Cette fonction n'est autre que l'implémentation de l'organigramme décrit dans la figure ci-dessus. Donc, on va passer à l'étude de la complexité de chacune des étapes de notre algorithme.

• Etape lecture du terrain : cette étape contient deux fonctions : une première permettant de lire la matrice de taille l*c à partir du fichier asc et une deuxième permettant de fixer la matrice de taille l*c à une matrice carrée de taille $2^n * 2^n$. La complexité de la première fonction est de O(l*c) alors que la deuxième fonction a une complexité de $O(p^2)$ avec p = 2n le plus petit multiple de 2 supérieur au maximum entre le nombre de lignes et le nombre de colonnes c. Donc la complexité globale de cette étape et de $O(p^2)$.

• Etape discrétisation du terrain : cette étape est composée d'une seule fonction récursive qui permet de discrétiser le terrain en appliquant la méthode des quadtree. La structure de récursivité et de type non-terminale car il y a des traitements à faire après les 4 appels récursifs de la fonction. La complexité de la récursivité l'emporte par rapport aux autres parties de la fonction, qui est de $O(nc.4^{nc})$ où nc est le nombre de cases dans la matrice.

• Etape affichage : Une seule fonction assure la partie de l'affichage du terrain. Il s'agit d'une seule boucle pour parcourir le tableau où sont stockés les résultats de la discrétisation. Dans cette boucle, on écrit dans un fichier les coordonnées des points NE (North East) et SW (South West) consitutant la diagonale de la case. Ce fichier nous permettre d'afficher la représentation des quadtrees comme dans les figures 2.1, 2.2, 2.3 et 2.4. Sa complexité est donc de O(cc) où cc est le nombre total de cases générées.

• Etape génération du graphe : l'étape génération du graphe est assurée par une seule fonction qui prend en entrée un certain nombre de paramètres. Ces paramètres sont les listes des cases vides et occupées, le nombre des cases vides et occupées, la liste contenant les informations de chaque case et le Radius Factor. Cette fonction permet d'établir toutes les connexions possibles entre les cases libres sur le terrain et de les stocker dans des valeurs. La complexité de cette fonction est de $O(fc^{fc})$.

• Etape stockage : Cette étape concerne le stockage des cases libres dans un fichier pour être utilisé par l'algorithme A*. Dans chaque ligne du fichier, il y a l'indice de la case dans la listes des cases libres et les coordonnées de son centre. Les cases sont ordonnées selon l'indice dans la liste des ouverts. Il s'agit d'une complexité de l'ordre de O(fc).

• Etape mesure du temps : Cette étape permet la mesure du temps d'exécution de l'outil à la fin de chaque étape de son exécution. Cette étape est de complexité de O(1) puisqu'il s'agit d'opérations de calcul et d'affectation.

Pour conclure, la complexité temporelle de notre outil est celle de l'étape de la génération du graphe, qui est de $O(fc^{fc})$ puisqu'elle représente la plus grande complexité de toutes les différentes étapes.
Pour valider notre analyse théorique, nous avons exécuté plusieurs fois notre outil sur le terrain des Pyrénées avec différentes configurations côté altitude et Radius Factor. A la fin de chaque exécution, nous collectons le temps d'exécution à chaque étape. Nous avons récupéré ces résultats qu'on va montrer dans le tableau ci-dessous. Nous notons les temps t_i avec i entre 1 et 5, les temps dédiés à chaque phase de notre exécution :

Altitude (m)	Radius Factor	T1 (ms)	T2(ms)	T3(ms)	T3(ms)	T5(ms)
500	3	145.298	19.942	29.012	11565.058	6.393
500	5	151.456	18.087	28.638	21456.680	6.198
1200	1.75	143.731	22.469	49.446	21191.302	10.978
1200	2	145.298	21.893	49.446	26489.672	11.267
1500	1.75	156.612	20.513	49.380	15872.880	9.406
1500	3	145.383	19.793	40.031	35776.032	9.308
1700	2	147.113	20.923	37.220	15243.578	8.220
2000	1.75	155.848	17.870	31.424	9910.796	7.376
2000	5	143.842	18.527	31.897	47038.384	7.592
2500	2	217.472	16.361	16.795	8070.006	4.702
2500	4.5	146.503	16.110	17.021	13994.952	4.780

Table 2.2 – Evolution du temps d'exécution des différentes phases de l'algorithme

A première vue, nous pouvons déjà remarquer que 90% du temps d'exécution s'est écoulé dans la 4 ème phase, c'est-à-dire, la génération du graphe horizontal. Les 10% n'ont pas subi de grandes variations avec le changement des configurations. Ceci s'explique par le fait que les phases s'appliquent sur tout le terrain et non pas sur les listes résultantes. Donc, on va se focaliser sur la 4ème étape de l'exécution. A partir de ces résultats, nous pouvons aussi remarquer la forte influence de la configuration en entrée sur le temps d'exécution de la génération du graphe. Nous pouvons en tirer deux constats :

- Plus on monte dans l'altitude, plus le temps diminue. Ceci s'explique par le fait que le nombre de cases diminue ce qui facilite les possibilités de recherche au niveau des graphes.
- Plus on augmente le Radius Factor, plus le temps augmente. En effet, plus le Radius Factor est grand, on peut explorer plus de voisins potentiels pour la case. Ceci rendra l'exploration des voisins plus lente.

On peut donc dire que l'impact de ces deux paramètres touchent essentiellement à la génération de graphe ce qui explique les variations du temps.

Pour prouver nos hypothèses, nous allons tracer l'évolution du temps d'exécution de la 4ème étape en fonction de l'altitude et du Radius Factor.

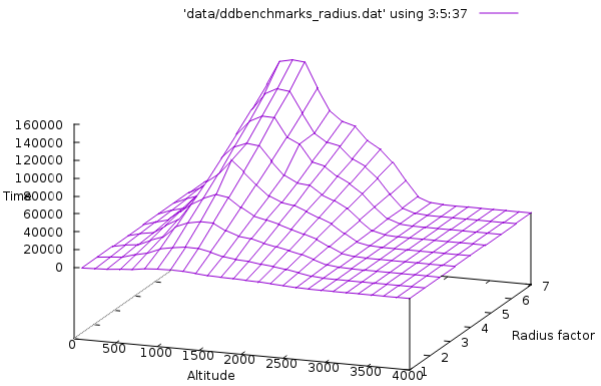


Figure 2.12 – Evolution du temps de génération du graphe en fonction de l'altitude et du Radius Factor

Nous remarquons que le pic du temps de génération de graphe est retrouvé à mi- altitude pour des Radius Factor assez importants. Ceci est dû au grand nombre d'arcs générés au niveau du graphe. On peut donc comprendre l'intérêt de l'optimisation au niveau temps. Ce constat nous amène aussi à faire l'étude de la complexité spatiale de notre algorithme pour voir l'étendu de l'utilisation de la mémoire par notre outil.

2.3.2 Etude de la complexité spatiale :

L'étude de la complexité concerne tout ce qui est étude de la mémoire allouée et la gestion de la pile. Dans cette partie, on s'intéresse plus aux types de variables, aux tailles des variables, aux méthodes de passages de paramètres (par adresse ou par valeur). Théoriquement, la tâche s'avère fastidieuse parce qu'il faut vérifier l'état de la mémoire à chaque instant.

Pour ce faire, nous avons déployé et intégré un module de mesure de la mémoire. Ce module permet de mesurer 4 paramètres principaux traduisant l'état de la mémoire :
- Total : Il traduit la taille mémoire globale utilisée par le programme
- Résident : c'est la mémoire allouée pour le processus et partagée avec les autres processus
- Texte : Il s'agit de la taille mémoire occupée par les lignes de codes écrites
- Données : C'est la taille de données dans la mémoire et la pile

Et donc, l'architecture de notre outil se présentera comme dans la figure ci-dessous

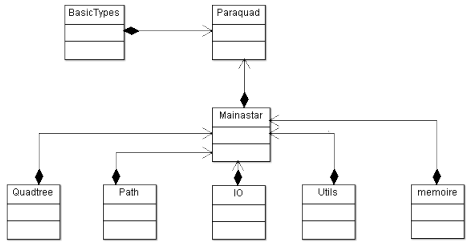


Figure 2.13 – La nouvelle architecture du projet

Pour des raisons d'ergonomie, nous avons réorganisé l'affichage de notre outil, pour rendre la mesure du temps et de la mémoire affichés à la fin de chaque phase. Comme a été fait dans la complexité temporelle, nous avons refait les mêmes tests effectués pour la complexité spatiale, mais, on suit cette fois le paramètre Data pendant les 5 phases de l'exécution de notre outil :

Altitude (m)	Radius Factor	Data 1	Data 2	Data 3	Data 4	Data 5
500	3	9637	11716	11716	11716	11716
500	5	9637	11716	11716	11716	11716
1200	1.75	9637	11716	11716	11716	11716
1200	2	9637	11716	11716	11716	11716
1500	1.75	9637	11716	11716	11716	11716
1500	3	9637	11716	11716	11716	11716
1700	2	9637	11716	11716	11716	11716
1700	4	9637	11716	11716	11716	11716
2000	1.75	9637	11716	11716	11716	11716
2000	5	9637	11716	11716	11716	11716
2500	2	9637	11716	11716	11716	11716
2500	4.5	9637	11716	11716	11716	11716

Table 2.3 – Quantité de la mémoire utilisée pour différentes configurations

Les valeurs sont calculées par une page de 4 Ko. On remarque que le niveau de mémoire est à son maximum au moment où on termine la deuxième phase de l'algorithme, qui est la phase de la discrétisation. Ceci est expliqué par le fait qu'on remplit les structures de données lors de l'application de la méthode des quadtrees. Donc, le fait d'optimiser la mémoire devient une nécessité afin d'alléger le taux utilisé.

2.4 Optimisation niveau temps d'exécution et mémoire :

Après avoir étudié la complexité de notre algorithme en termes de temps et de mémoire, nous allons procéder à l'opération de l'optimisation de notre algorithme. Cette opération consiste essentiellement à :
- Faire les allocations mémoires aves les tailles : on n'alloue que la taille nécessaire pour nos structures de données.
- Libérer les allocations mémoires lorsque l'algorithme n'aura plus besoin d'elles
- Eliminer les variables inutilisées
- Réduire le jeu d'instructions

Et donc, ceci nous amène à réduire les coûts de notre algorithme au niveau temps. Pour évaluer les résultats, nous allons reprendre les mêmes tableaux réalisés pour l'étude temporelle et spatiale avec la version optimisée de l'outil.

Altitude (m)	Radius Factor	T1 (ms)	T2(ms)	T3(ms)	T4(ms)	T5(ms)
500	3	154.949	9.698	58.373	8183.791	11.623
500	5	164.687	9.606	58.470	16437.212	11.514
1200	1.75	149.294	14.165	105.403	14754.281	22.831
1200	2	172.555	13.641	104.498	17702.696	22.849
1500	1.75	148.665	12.489	86.939	11854.218	19.324
1500	3	185.155	12.263	90.363	27637.494	19.446
1700	2	180.894	11.307	77.177	12195.030	17.439
1700	4	154.577	12.210	78.252	34722.064	17.296
2000	1.75	154.773	10.214	72.048	9750.385	16.062
2000	5	147.553	10.238	68.664	43181.936	15.742
2500	2	187.820	8.181	36.463	4888.911	9.853
2500	4.5	187.820	8.611	36.939	16111.051	9.915

Table 2.4 – Evolution du temps des différentes phases de l'algorithme dans la version optimisée

Comme on a fait pour la partie analyse temporelle, nous allons tracer le graphe traduisant l'évolution du temps consacré à la génération du graphe en fonction de l'altitude et du Radius Factor.

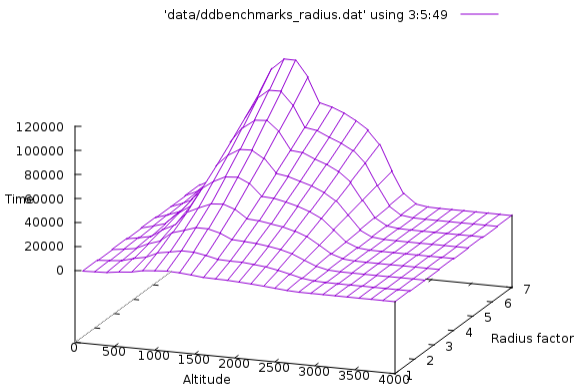


Figure 2.14 – Courbe de l'évolution du temps de génération du graphe dans la version optimisée

Pour l'optimisation mémoire, le tableau ci-dessous montre les quantités de la mémoire allouées pour les données lors des exécutions de notre outil.

Altitude (m)	Radius Factor	Data 1	Data 2	Data 3	Data 4	Data 5
500	3	7657	7657	7364	7071	2188
500	5	7657	7657	7364	7071	2188
1200	1.75	7657	7657	7364	7071	2188
1200	2	7657	7657	7364	7071	2188
1500	1.75	7657	7657	7364	7071	2188
1500	3	7657	7657	7364	7071	2188
1700	2	7657	7657	7364	7071	2188
1700	4	7657	7657	7364	7071	2188
2000	1.75	7657	7657	7364	7071	2188
2000	5	7657	7657	7364	7071	2188
2500	2	7657	7657	7364	7071	2188
2500	4.5	7657	7657	7364	7071	2188

Table 2.5 – Quantité de la mémoire allouée pendant chaque phase de l'algorithme optimisé

A partir de ces résultats obtenus, on peut conclure qu'on a réussi l'opération de l'optimisation. Le temps d'exécution a été réduit de 25% après avoir calculé tous les rapports entre les anciens temps et les nouveaux temps de génération de graphes. La mémoire a été optimisée de 35% grâce aux actions faites sur notre algorithme.

2.5 Conclusion :

Dans ce chapitre, nous avons effectué une analyse complète de notre outil. Cette analyse a été faite sur deux niveaux : le premier niveau concerne l'analyse de son mode de fonctionnement alors que le deuxième niveau concerne une analyse de la complexité temporelle et spatiale. Ensuite, nous avons traité la phase d'optimisation de l'exécution de notre outil. Tout ça a été fait pour répondre à la première problèmatique, qui consistait à améliorer les performances de l'outil et réduire les coûts. Nos actions d'optimisations nous ont permis de réduire les coûts mémoire et temps de 30%. Nous allons maintenant assurer le passage de la version 2D vers la 3D, qui sera l'objectif du prochain chapitre de ce rapport.

Chapitre 3

MISE EN PLACE DE L'OUTIL DE PLANIFICATION DE VOL EN 3D

Sommaire

3.1 Introduction 41
3.2 Analyse de l'existant : Les Octrees 41
3.2.1 Avantages et Inconvénients des Octrees : 42
3.2.2 Principe de fonctionnement des Octrees : 42
3.2.3 Conclusion : 45
3.3 Solution proposée : 45
3.3.1 Principe de la solution : 45
3.4 Test & Résultats : 48
3.4.1 Résultats sur le terrain de l'Andorre : 48
3.4.2 Résultats sur le terrain des Pyrénées : 49
3.4.3 Résultats sur une falaise : 50
3.5 Etude de la complexité temporelle et spatiale : 52
3.5.1 Etude de la complexité temporelle : 52
3.5.2 Etude de la complexité spatiale : 55
3.6 Conclusion : 56

3.1 Introduction

Après avoir fait le tour sur la version 2D de notre outil, nous passons maintenant à la génération de version 3D de notre outil. Générer la version 3D nous permettra de planifier un chemin suivant des déplacements horizontaux et verticaux. Ceci faciltera l'exploration par la suite à travers l'algorithme A*. Ce chapitre sera divisé en trois parties. La première partie sera dédiée à l'analyse de l'existant en terme de discrétisation 3D et générateur de graphes. Quant à la deuxième partie, c'est la partie dans laquelle nous allons exposer la solution proposée ainsi que son déploiement. Finalement, nous allons clôturer ce chapitre par une étude de complexité temporelle et spatiale de la nouvelle version.

3.2 Analyse de l'existant : Les Octrees

La méthode existante pour la représentation des terrains et le calcul des plans de vol en 3D est la méthode des octrees. Les octrees sont en effet une structure de données de type arbre dont chaque nœud peut avoir jusqu'à 8 fils. Ils sont généralement utilisés pour discrétiser un espace tridimensionnel en le subdivisant d'une manière récursive en huit octants. Cette méthode est généralement utilisée pour les cas d'applications suivants :

- L'indexation spatiale
- La détection de collision dans le cadre de la 3D
- l'élimination des objets hors du cône de vue dans le cadre d'un rendu 3D
- l'observateur d'état

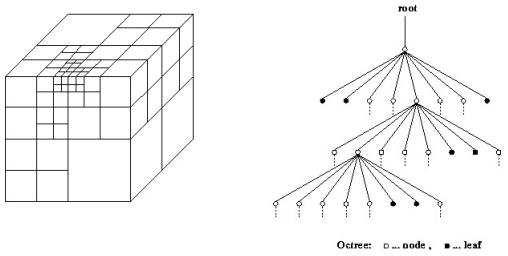


Figure 3.1 – Un exemple d'un octree

« Les Octrees ont été indépendamment développés par plusieurs chercheurs. Hunter les a mentionnés comme étant une extension naturelle des quadtrees. Reddy et Rubin ont men-

tionné les octrees comme étant une parmi trois méthodes pour la représentation des objets solides. La seconde est une généralisation tridimensionnelle du quadtree point de Finkel et Bentley, c'est-à-dire une décomposition en des parallélépipèdes (contrairement aux cubes) avec des plans perpendiculaires aux axes x,y et z. La troisième divise un objet en des parallélépipèdes rectangulaires d'une taille arbitraire qui ne sont pas nécessairement alignés avec un axe. » [21]

3.2.1 Avantages et Inconvénients des Octrees :

Les octrees est par excellence la méthode la plus adaptée pour la représentation tridimensionnelle des espaces. Néanmoins, elle a ses avantages et ses inconvénients. On peut résumer les avantages et les inconvénients des Octrees dans le tableau suivant :

Avantages	Inconvénients
L'arbre généré est pré-calculé, ce qui rend les performances de la machine concentrées sur son utilisation	La subdivision n'est pas optimale dans les zones fermées
La méthode de discrétisation rend le repérage facile dans l'espace	Consommation de mémoire énorme
L'algorithme est facile à implémenter	Temps de calcul important
Un rendu graphique avec plein de détails	La pixellisation lorsqu'on change le zoom

Table 3.1 – Avantages et Inconvénients des Octrees

3.2.2 Principe de fonctionnement des Octrees :

Les octrees fonctionnent de la manière suivante : « Etant donnée une matrice de voxels de taille 2^n x 2^n x 2^n, une région octree est construite en subdivisant d'une manière répétitive un des octants, sous-octants ... jusqu'à obtenir des blocks qui contiennent une seule valeur. Ce processus est généralement représenté par un arbre de degré 8 où la racine correspond à la matrice entière et les 8 fils du nœud racine correspondent aux octants UNW, UNE, USW, USE, DNW, DNE, DSW et DSE. Les nœuds feuilles correspondent aux blocs de la matrice où la subdivision n'est plus nécessaire. Les nœuds d'un niveau k représentent des blocs de taille 2^k x 2^k x 2^k et ils sont souvent désignés comme des nœuds de taille 2^k. Ainsi, un seul

nœud au niveau 0 correspond à un simple voxel dans la matrice des voxels alors que le seul nœud au niveau n correspond à la racine de la région octree. » [15]

Les deux figures ci-dessous représentent un exemple de la discrétisation d'un objet solide avec la méthode des octrees ainsi que l'arbre généré.

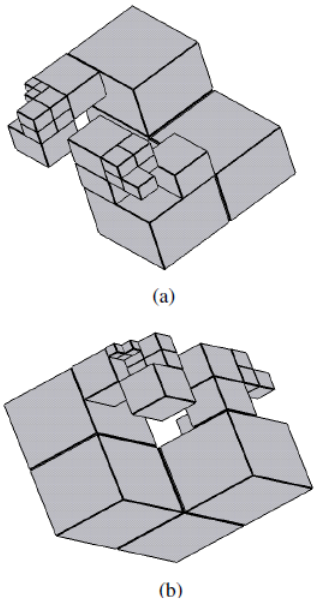


Figure 1: Two orthogonal projections of the region octree decomposition of a sample object.

Figure 3.2 – La discrétisation d'un objet par les octrees[15]

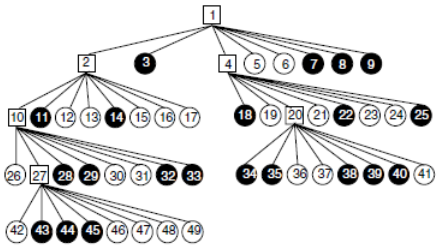


Figure 2: The tree representation of the region octree block decomposition corresponding to the sample object in Figure 1. The internal nodes are depicted as white squares, while the WHITE and BLACK leaf nodes are depicted as white and black circles, respectively. The nodes are referenced in the discussion by their numeric label.

Figure 3.3 – L'arbre généré par la discrétisation de l'objet[15]

Dans notre cas d'application, nous allons appliquer les octrees type MX sur nos terrains. On prend la matrice d'altitude à partir du fichier asc, et on commence à la parcourir. Au début, on considère le terrain comme un grand cuboïde. A chaque fois qu'on retrouve un cuboïde dont l'altitude est supérieur à 150m, on le subdivise en 8 octrees et on ajoute les octrees résultants à la liste des cuboïdes. Cette opération se répète jusqu'à obtenir des octrees d'altitude 150m, qui est la valeur du palier demandé par l'industriel. Après, on commence à différencier les octrees libres et les octrees occupés, qui seront considérés comme des obstacles, dans la liste résultante.

Les figures suivantes montreront les résultats de la discrétisation avec la méthode des octrees sur les terrains d'Andorre, de la région de Grenoble et des Pyrénées.

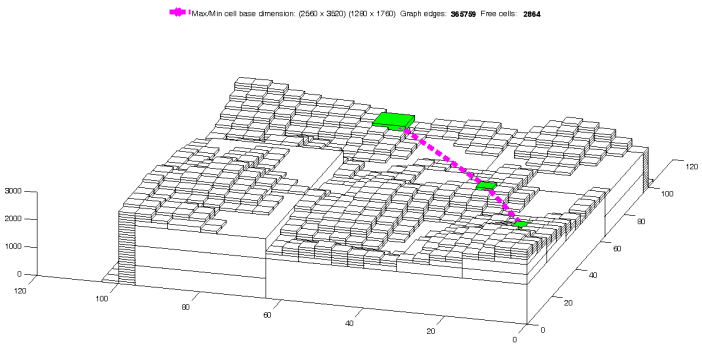


Figure 3.4 – L'Andorre représenté par les octrees

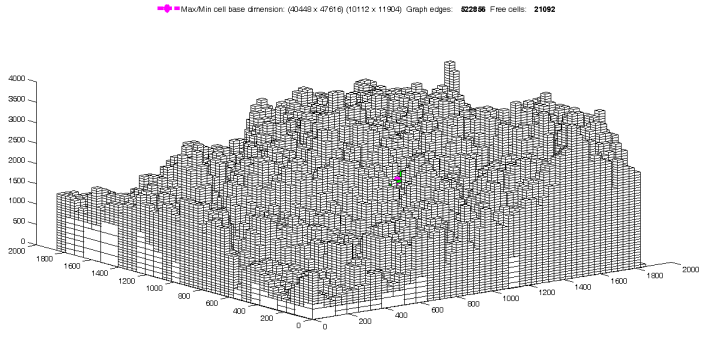


Figure 3.5 – La région de Grenoble représentée par les octrees

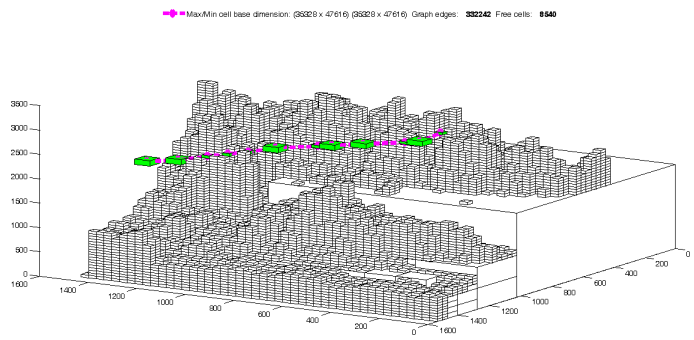


Figure 3.6 – Les Pyrénées représentés par les octrees

Il est à noter aussi que notre l'algorithme A* fonctionne sur les octrees que sur les quadtrees. En effet, les cases colorées en vert sur l'Andorre et les Pyrénées ont été sélectionnées par l'algorithme A* afin de servir de points de passages pour le chemin optimal

3.2.3 Conclusion :

Malgré les avantages séduisants de la méthode des octrees, le fait qu'elle soit gourmande en matière de temps d'exécution et de mémoire ne satisfait pas les exigences demandées. Donc, nous sommes dans l'obligation de chercher une nouvelle méthode pour la construction de la version 3D de notre outil. De ce fait, la prochaine section sera consacrée à l'explication de la solution proposée pour remédier aux problèmes de l'octree.

3.3 Solution proposée :

3.3.1 Principe de la solution :

La solution qu'on a proposée consiste à développer un nouveau module dans notre version 2D pour la rendre compatible avec le contexte 3D. En effet, l'idée vient du fait qu'on est parvenu à trouver un chemin de vol sur une altitude donnée en utilisant les quadtrees. Donc, on a pensé à généralisé la méthode sur toutes les altitudes du terrain, c'est-à-dire faire la discrétisation et la recherche du graphe sur tous les paliers du terrain. Et donc, l'algorithme devra faire la discrétisation chaque 150m et rechercher dans le quadtree de chaque palier le

graphe complet. Donc, on n'a plus à lire l'altitude en entrée, on n'a besoin que du Radius Factor ainsi que du nom du fichier asc. Le fait de générer les graphes sur chaque palier donnera naissance à des graphes « horizontaux » suivant les paliers. Ensuite, on passe au raccordement des niveaux en créant des arcs verticaux entre deux paliers successifs. La création des arcs verticaux se fait de la manière suivante : Sur un palier, on commence à chercher les points nous permettant de monter au palier supérieur. Après, on fait une sélection parmi les points calculés pour éliminer ceux qui vont donner naissance à des arcs traversant des obstacles. Une fois la sélection faite, on calcule les arcs entre les deux paliers et on passe au palier suivant. L'industriel a exigé que l'angle de montée maximal soit de 15 degrés à cause des contraintes imposées par le modèle dynamique de l'hélicoptère. La finalité de cette procédure est un ensemble de graphes « verticaux » entre chaque couple de paliers. Le nombre de graphes générés dépendra fortement du nombre de paliers calculés dans le terrain. On répète la même procédure jusqu'à atteindre le dernier palier correspondant à l'altitude maximale détectée sur le terrain. Au final, notre outil produira un graphe complet contenant l'ensemble des graphes horizontaux et verticaux de chaque niveau. Il s'agit donc de développer un nouveau module de calcul pour raccorder les quadtrees horizontaux et de généraliser la méthode 2D sur tous les paliers possibles.

Ainsi, le processus de fonctionnement de notre algorithme sera décrit par la figure ci-dessous :

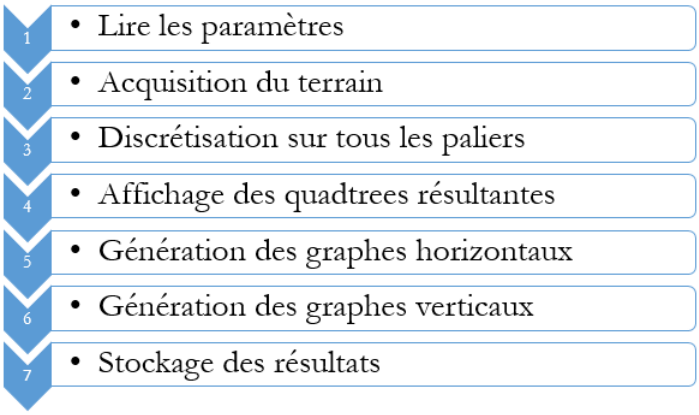


Figure 3.7 – Le processus de l'outil en 3D

Ceci peut être traduit par l'organigramme décrit par la figure ci-dessous :

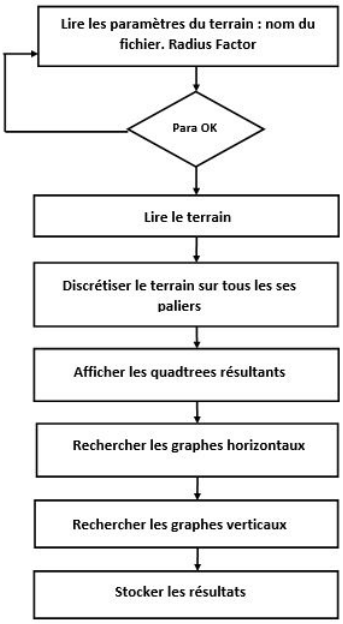


Figure 3.8 – Organigramme de l'outil en 3D

Le diagramme de classe correspondant à notre outil est représenté par la figure ci-dessous :

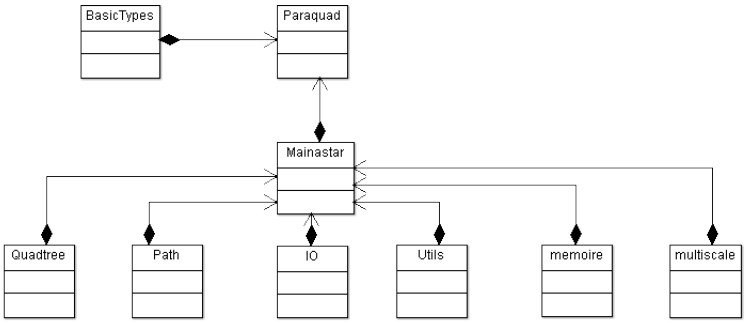


Figure 3.9 – Diagramme de classe de la version 3D de l'outil

Le fichier multiscale contient toutes les fonctions permettant la création du graphe vertical entre les niveaux.

3.4 Test & Résultats :

3.4.1 Résultats sur le terrain de l'Andorre :

Nous avons appliqué notre outil sur le terrain de l'Andorre avec la configuration suivante : Radius Factor : 2.75, Pas entre les centres : 1 Km, Angle de montée minimale 5,7 degrés qui correspond à un arc de valeur 1500 m. Cette configuration a produit les graphes représentés par les deux figures ci-dessous :

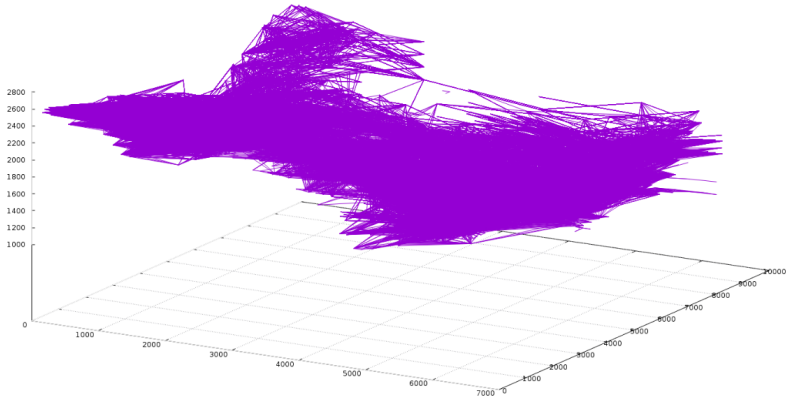


Figure 3.10 – La vue complète du graphe sur l'Andorre

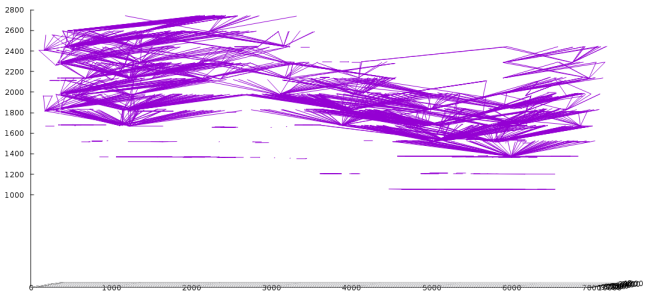


Figure 3.11 – La vue à côté du graphe

On remarque que le graphe possède une grande connexité sur le plan horizontal (figure 33) grâce à la valeur importante du Radius Factor. Cette valeur nous permet d'avoir un voisinage riche pour chaque case libre. Sur le plan vertical, on constate que le raccordement est assuré entre les niveaux du terrain, ce qui nous permet de monter en altitude lorsque le chemin l'impose.

3.4.2 Résultats sur le terrain des Pyrénées :

Nous avons exécuté notre outil sur le terrain des Pyrénées avec la configuration suivante : Radius Factor : 1.75, Pas entre les centres : 1 Km, Angle de montée minimale 5.7 degrés correspondant à un arc maximale de valeur 1500 m.

Ceci a donné naissance à un graphe volumineux qu'on ne peut pas tracer entièrement. Et donc, nous avons pensé à tracer un morceau du graphe sur une zone déterminée par des coordonnées X et Y bien précises. Ce graphe sera représenté par les deux figures ci-dessous, la première représente la vue 3D du terrain et la deuxième représente une vue de côté pour voir les arcs verticaux. Les figures représentent la portion du terrain donc X et Y sont entre 50000 et 70000 m :

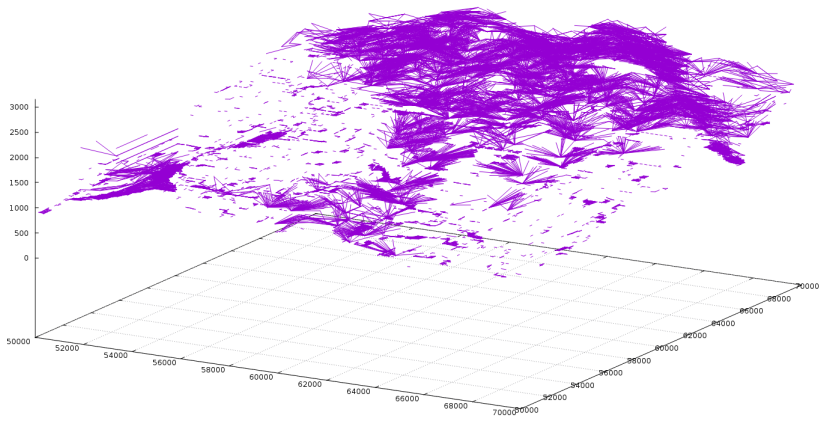


Figure 3.12 – La vue 3D du graphe sur les Pyrénées

La figure ci-dessous ne montrera que les arcs verticaux générés par l'outil.

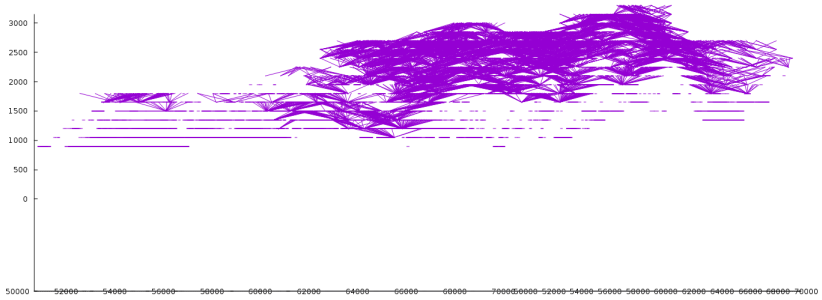


Figure 3.13 – La vue à côté du graphe

On remarque que la connexité horizontale du graphe est faible par rapport au graphe généré sur le terrain de l'Andorre. De ce fait, on peut comprendre l'importance du Radius Factor dans la génération du graphe horizontal. D'autre part, la densité du graphe vertical revient au fait que le terrain est très dense. Et donc, la probabilité de rejoindre le palier suivant devient plus importante.

3.4.3 Résultats sur une falaise :

Afin de consolider les performances de notre outil, nous l'avons testé sur un terrain généré. Le terrain comporte une falaise d'altitude 1500m. Le but était de de voir si l'outil peut générer un graphe permettant un vol d'oiseau au-dessus de la falaise. La configuration mise en place est la suivante :

- Radius Factor : 2.75.
- Pas entre les centres : 1 Km.
- Angle de montée minimale 5,7 degrés qui correspond à un arc de valeur 1500.

Ceci a donné le graphe représenté par les figures ci-dessous. La première figure représente la vue 3D alors que la deuxième figure représente une vue de dessus du terrain :

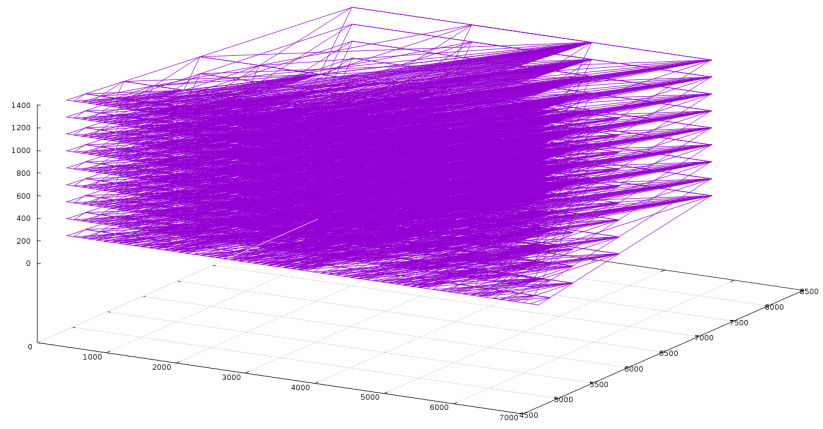


Figure 3.14 – La vue complète du graphe sur la falaise

La vue de dessus nous permet de voir comment l'algorithme permet de passer la falaise.

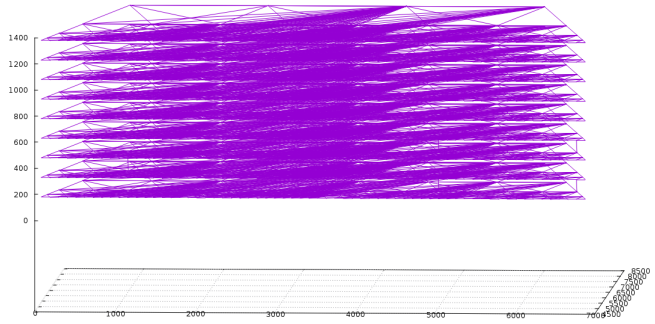


Figure 3.15 – La vue du dessus du graphe

On remarque que l'outil a pu relier les cases pour assurer le vol d'oiseau. Mais, la connexité reste encore faible pour pouvoir acheminer les cases ensemble. La solution consiste à exécuter l'outil sur le terrain de la falaise avec un radius factor plus important comme 4 ou 5 afin de pouvoir obtenir un voisinage important sur le plan horizontal.

3.5 Etude de la complexité temporelle et spatiale :

Nous allons maintenant procéder à l'étude de complexité de notre outil. Cette étude sera faite sur le volet temporel et sur le volet spatial. Il est à noter que cette version est basée sur la version 2D optimisée.

3.5.1 Etude de la complexité temporelle :

L'étude de la complexité algorithme sera comme celle établie pour la version 2D. Elle sera donc une étude « top down », c'est-à-dire qu'on va commencer par le niveau le plus haut pour ensuite étaler l'étude sur les niveaux les plus bas.

En se basant sur notre diagramme de classes établi dans le chapitre précédent, nous commençons notre étude sur le fichier paraquad contenant la fonction main. Ensuite, l'étude va entrer dans les détails de l'algorithme.

- Dans le fichier paraquad, la complexité reste inchangée. Elle est de l'ordre de O(1) à laquelle on ajoute la complexité de la fonction AstarPathPreprocessing. Donc, on va s'intéresser à la complexité de cette fonction qui nous donnera la complexité globale de notre outil.
- Pour étudier la fonction AstarPathPreprocessing, il faut accéder au fichier mainastar qui contient l'implémentation de cette fonction. Cette fonction n'est autre que l'implémentation de l'organigramme décrit ci-dessus. Donc, on va passer à l'étude de la complexité de chacune des étapes de notre algorithme.

• Etape lecture du terrain : Lors de l'opération d'optimisation, on a simplifié cette étape. Elle contient désormais une seule fonction permettant de lire la matrice de taille l*c à partir du fichier asc. La complexité de cette étape est de O(l*c) puisqu'il s'agit d'une fonction ayant deux boucles imbriquées.

• Etape discrétisation du terrain : cette étape est composée d'une seule fonction récursive qui permet de discrétiser le terrain en appliquant la méthode des quadtree. Cette fonction est appelée n fois dans une boucle où n est le nombre de paliers. La complexité de la récursivité n'a pas changé entre la version 2D et la version 3D. Elle est toujours de $O(nc.4^{nc})$ où nc est le nombre de cases dans la matrice. En ajoutant la complexité de la boucle, la complexité de

cette étape devient de l'ordre de $O(n.nc.4^{nc})$.

• Etape affichage : Une seule fonction appelée n fois assure la partie de l'affichage du terrain. Il s'agit d'une seule boucle pour parcourir le tableau où sont stockés les résultats de la discrétisation partiellement. En effet, chaque palier est délimité par une case de départ et une case de fin. La complexité de cette étape est donc de O(n.cc) où cc est le nombre total de cases générées et n le nombre de paliers.

• Etape génération du graphe horizontal : Il s'agit de l'étape génération du graphe au niveau de la version 2D à laquelle on a ajouté un appel itératif selon les paliers. La complexité de cette étape est donc la complexité de la fonction multipliée par celle de la boucle. Elle est donc de $O(n.fc^{fc})$.

• Etape génération du graphe vertical : Cette étape est composée de trois fonctions appelées n-1 fois. La première fonction calcule les points de montées correspondants à des centres de cases sur le palier. Sa complexité est de O(px.py) où px et py sont les pas suivant x et y (Généralement on choisit le même pas sur les deux axes). La deuxième fonction est une fonction d'affichage permettant de montrer les centres de cases par gnuplot. Sa complexité est de O(cc) avec cc le nombre de cases dans le palier. Quant à la troisième fonction, elle permet de calculer les arcs selon les contraintes de l'angle de montée. Sa complexité est de O(sc.fc) avec sc le nombre de points dans le palier actuel et fc le nombre de cases libres dans le palier suivant. La complexité de la troisième fonction l'emporte sur les deux autres fonctions. Donc, la complexité de l'étape est de O ((n-1).sc.fc))

• Etape stockage : Cette étape concerne le stockage des cases libres dans un fichier pour pouvoir être utilisé par l'algorithme A* appelée n fois. Il s'agit d'une complexité de l'ordre de O(n.fc).

Pour conclure, la complexité temporelle de notre outil est celle de l'étape de la génération du graphe horizontal sur les paliers, qui est de $O(n.fc^{fc})$ puisqu'elle représente la plus grande complexité de toutes les différentes étapes.

Côté temps d'exécution, nous avons constaté que le temps d'exécution est fortement influencé par 3 paramètres

- Le Radius Factor : pour le calcul du graphe horizontal

- Le pas entre les centres des cases : pour calculer les points de montée
- L'angle de montée : C'est une contrainte imposée par l'industriel entre 0 et 15 degrés

Pour valider notre hypothèse, nous avons exécuté plusieurs fois notre outil sur le terrain des Pyrénées avec différentes configurations Radius Factor, pas et Angle de montée. Pour l'angle de montée, la valeur 800 correspond à un angle de 10.61 degrés, la valeur 1500 correspond à un angle de 5.7 degrés et la valeur 1800 correspond à un angle de montée de 4.76 degrés. Nous avons récupéré ces résultats qu'on va montrer dans le tableau ci-dessous. Nous notons les temps t_i avec i entre 1 et 6, les temps dédiés à chaque phase de notre exécution.

Radius Factor	Pas	Angle de montée	T1(ms)	T2(ms)	T3(ms)	T4(ms)	T5(ms)	T6(ms)
1.75	200	800	163.704	248.889	1281.950	2255004.928	785186.994	341.572
1.75	200	1500	161.610	249.988	1378.727	2337588.224	858089.216	298.495
1.75	200	1800	155.660	223.470	1294.127	2367280.640	886035.008	293.031
1.75	500	800	157.308	228.554	1248.938	2352575.232	976542.384	309.032
1.75	500	1500	166.788	226.299	1248.768	2348464.640	100206.112	295.241
1.75	500	1800	158.435	292.518	1332.856	2336820.736	105029.096	286.525
1.75	1000	800	151.457	216.887	1263.669	2326065.152	25545.596	300.684
1.75	1000	1500	167.424	204.126	1202.846	2277404.672	25845.016	292.889
1.75	1000	1800	158.400	254.270	1371.937	2341115.904	27254.022	294.125
2	200	800	154.122	224.927	1240.796	2935980.544	823938.496	288.819
2	200	1500	174.310	241.845	1299.012	2921665.024	845856.384	390.252
2	200	200	160.423	225.698	1253.435	2927017.216	872963.456	322.500
2	500	800	152.490	226.837	1247.115	2928113.152	95827.096	289.739
2	500	1500	149.452	216.292	1246.760	2888950.016	100054.432	297.626
2	500	1800	157.553	252.091	1330.861	2870661.632	101068.232	282.613
2	1000	800	154.459	205.564	1227.758	279894.336	25715.718	275.888
2	1000	1500	159.160	248.759	1330.833	290812.016	26396.052	279.110
2	1000	1800	146.458	216.398	1187.082	2891845.888	26712.488	284.782

Table 3.2 – Analyse du temps d'exécution de l'outil en 3D

Ces essais ont confirmé nos analyses. D'une part, le temps le plus consommé est dans la génération du graphe horizontal. D'autre part, plus l'angle est petit (la valeur est grande), plus on prend du temps dans la phase de la génération du graphe vertical. Ceci est dû au fait que lorsqu'on réduit l'angle, on fait augmenter l'intervalle de recherche, qui est entre 15

degrés et l'angle introduit ce qui donne plus de possibilités de calcul à l'algorithme.

3.5.2 Etude de la complexité spatiale :

Pour l'étude de la complexité spatiale, nous avons gardé la même méthode utilisée pour l'analyse de la version 2D de notre outil. Nous avons recueilli les valeurs de la mémoire totale, Résident, Texte et données comme on a fait précédemment. Nous allons nous intéresser aux résultats de la quantité mémoire allouée aux données en fonction du Radius Factor, pas et angle de montée. Les résultats sont collectés dans le tableau ci-dessous.

Radius Factor	Pas	Angle de montée	Data	Data	Data	Data	Data	Data
1.75	200	800	72502	72502	66642	66642	2188	2188
1.75	200	1500	72502	72502	66642	66642	2188	2188
1.75	200	1800	72502	72502	66642	66642	2188	2188
1.75	500	800	72502	72502	66642	66642	2188	2188
1.75	500	1500	72502	72502	66642	66642	2188	2188
1.75	500	1800	72502	72502	66642	66642	2188	2188
1.75	1000	800	72502	72502	66642	66642	2188	2188
1.75	1000	1500	72502	72502	66642	66642	2188	2188
1.75	1000	1800	72502	72502	66642	66642	2188	2188
2	200	800	72502	72502	66642	66642	2188	2188
2	200	1500	72502	72502	66642	66642	2188	2188
2	200	200	72502	72502	66642	66642	2188	2188
2	500	800	72502	72502	66642	66642	2188	2188
2	500	1500	72502	72502	66642	66642	2188	2188
2	500	1800	72502	72502	66642	66642	2188	2188
2	1000	800	72502	72502	66642	66642	2188	2188
2	1000	1500	72502	72502	66642	66642	2188	2188
2	1000	1800	72502	72502	66642	66642	2188	2188

Table 3.3 – Quantité mémoire allouée aux données par l'outil

La phase de discrétisation garde encore la quantité mémoire la plus allouée par l'outil au cours de son exécution. En effet, à ce moment-là, on alloue toute la mémoire nécessaire pour le bon fonctionnement de notre algorithme. Cette allocation doit supporter l'allocation du

terrain, tous les paliers crées par la discrétisation et le remplissage des listes des cases vides et occupés pour pouvoir les utiliser ensuite par l'algorithme A*. Ensuite, on commence à libérer petit à petit la mémoire pour alléger la pile.

3.6 Conclusion :

Au cours de ce chapitre, nous avons explicité la méthode avec laquelle on a travaillé pour déployer la version 3D de notre outil. Nous avons fait un état de l'art sur ce qui existe déjà et nous avons tiré nos conclusion sur l'application des méthodes existantes dans notre cas. Puis, nous avons proposé une solution alternative à l'existant et nous avons détaillé les étapes de la conception, du développement et du déploiement. Notre solution consiste essentiellement à appliquer les quadtrees sur des paliers de 150m et à stocker les informations de chaque case dans les structures de données adéquates. Le résultat devrait être un graphe 3D reliant les différentes zones du terrain. Ceci a été vérifié et validé suite à une phase de test de notre outil. Finalement, nous avons procédé à une étude de complexité côté temps et mémoire pour voir les performances résultantes. Maintenant, nous passons à l'implémentation embarquée de notre outil, chose qui sera traitée dans le chapitre suivant.

Chapitre 4

IMPLEMANTATION EMBARQUEE

Sommaire

4.1 Introduction 58
4.2 Présentation de l'environnement embarqué : 58
4.2.1 Présentation de la plateforme : 58
4.2.2 Présentation de la partie AP SOC : 61
4.2.2.1 Présentation de la partie Logique programmable : 62
4.2.2.2 Présentation de la partie processeur : 62
4.3 Implémentation embarquée sur la carte Zybo 64
4.3.1 Installation de Xilinux : 65
4.3.2 Paramétrage de la carte : 66
4.3.3 Mise à niveau de la carte : 66
4.3.4 Création de l'environnement de développement : 67
4.3.5 Téléversement du code vers la carte : 67
4.4 Analyse des résultats : 67
4.4.1 Analyse côté temps d'exécution : 67
4.4.2 Analyse côté mémoire : 70
4.5 Conclusion : 71

4.1 Introduction

Dans le chapitre précédent, nous avons mis en place la version 3D de notre outil, celle qui sera utilisée pour la planification de vol par notre client. Nous allons maintenant tester notre outil sur un environnement embarqué afin de simuler les conditions de vol en temps réel. Ce chapitre sera réparti sur trois volets. Dans un premier temps, nous allons présenter l'environnement sur lequel on va travailler et détailler ses caractéristiques techniques. Ensuite, nous étalerons les étapes de l'implémentation. Finalement, nous ferons une analyse des performances de la carte lors de l'exécution.

4.2 Présentation de l'environnement embarqué :

4.2.1 Présentation de la plateforme :

Le choix de l'environnement embarqué s'est porté sur la Zybo Zynq 7000, produit de Digilent.

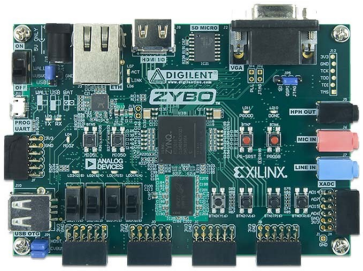


Figure 4.1 – La carte Zybo Zynq 7000

Le Zybo (Zynq Board) est une plateforme de développement de logiciels embarqués et de circuits numériques d'entrée de gamme, riche en fonctionnalité et prête à l'emploi. Elle est construite à base du Z-7010, le plus petit membre de la famille Xilinx Zynq-7000. Le Z-7010 est conçu selon l'architecture All Programmable System On Chip de Xilinx (AP SOC), qui intègre un processeur Dual-Core, un ARM Cortex A9 et une logique FPGA de la série 7 de Xilinx. Lorsqu'il est couplé avec un ensemble riche de périphériques multimédia et de connectivité disponibles sur le ZYBO, le Z-7010 peut supporter la conception entière d'un

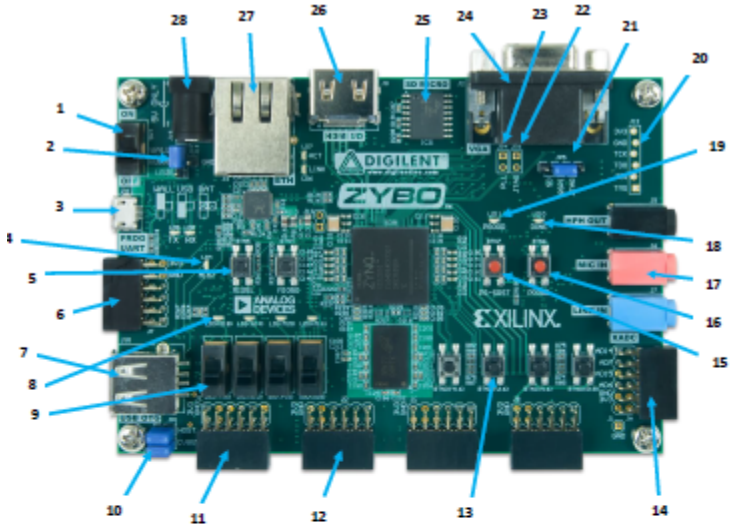
système. Les mémoires à bord, les E/S vidéo et audio, les ports USB double-rôle, Ethernet, la porte SD rendra votre conception prête sans aucun matériel ajouté. De plus, six ports Pmod sont disponibles pour faciliter l'agrandissement de toute conception. [9]
Le tableau ci-dessous regroupe les principales caractéristiques de la carte :

Processeur	A9 Cortex dual-core de fréquence 650 MHz
Mémoire	DDR3 512 Mo x32 avec 8 canaux de DMA
Contrôleur de périphériques à haute bande-passante	1G EthernetTrimode, USB 2.0, SDIO (supporte le système de fichiers Linux)
Contrôleur de périphériques à basse bande-passante	SPI, UART, CAN, I^2C
Logique Reprogrammable	Equivalente à la FPGA Artix-7
Partie logique	4400 parties
RAM de la FPGA	240 Ko
Horloge interne de la FPGA	450 MHz
Partie DSP de la FPGA	80 parties
Gestionnaire d'horloge de la FPGA	2
Convertisseurs de la FPGA	XADC
Mémoire EEPROM	Externe programmée sur 48 bits
Interface série	Une interface Flash en écriture sur 128Mo
Les pins GPIO	6 pushbuttons, 4 slide switches, 5 LEDs
Les portes Pmod	Six : 1 dédiée au processeur, 1 analogique/numérique, 3 différentielles à grande vitesse 1 dédiée à la logique reprogrammable
Test	JTAG et un convertisseur UART / USB

Table 4.1 – Les caractéristiques de la carte Zybo[9]

Le Zybo est compatible avec la nouvelle version de Vivado Design Suite haute performance ainsi que la boîte à outil ISE/EDK. Ces boîtes à outils mélangent la conception logique de la FPGA avec le développement logiciel embarqué ARM dans un flux de conception facile à utiliser et intuitif. Ils peuvent être utilisés pour la conception de systèmes de toute complexité, d'un système d'exploitation en cours d'exécution complète plusieurs applications serveur en tandem, jusqu'à un programme simple en métal nu qui contrôle certains voyants. [9]

La figure ci-dessous présente les différents composants de Zynq Zybo ainsi que leur emplacement sur la plateforme :



Callout	Component Description	Callout	Component Description
1	Power Switch	15	Processor Reset Pushbutton
2	Power Select Jumper and battery header	16	Logic configuration reset Pushbutton
3	Shared UART/JTAG USB port	17	Audio Codec Connectors
4	MIO LED	18	Logic Configuration Done LED
5	MIO Pushbuttons (2)	19	Board Power Good LED
6	MIO Pmod	20	JTAG Port for optional external cable
7	USB OTG Connectors	21	Programming Mode Jumper
8	Logic LEDs (4)	22	Independent JTAG Mode Enable Jumper
9	Logic Slide switches (4)	23	PLL Bypass Jumper
10	USB OTG Host/Device Select Jumpers	24	VGA connector
11	Standard Pmod	25	microSD connector (Reverse side)
12	High-speed Pmods (3)	26	HDMI Sink/Source Connector
13	Logic Pushbuttons (4)	27	Ethernet RJ45 Connector
14	XADC Pmod	28	Power Jack

Table 1. ZYBO Device Diagram.

Figure 4.2 – Organisation de la plateforme[9]

Nous allons maintenant nous intéresser à la partie processeur / FPGA qui est dans le composant AP SOC. En effet, c'est sur cette partie que nous allons implémenter notre outil software pour ensuite le tester dans les conditions temps réel.

4.2.2 Présentation de la partie AP SOC :

Il s'agit du composant essentiel de la carte Zybo Zynq. Il est le processeur qui gère toutes les opérations au niveau de l'environnement. C'est sur ce composant que nous allons implémenter notre outil software. L'AP SOC est constitué de deux sous-systèmes totalement indépendants : Le système processeur basé sur l'architecture ARM et le système logique programmable dont le fonctionnement est similaire à une carte FPGA.
La figure suivante décrit l'architecture de l'AP SOC en montrant les étendues de chaque sous-système ainsi que les liens de connexions entre les deux :

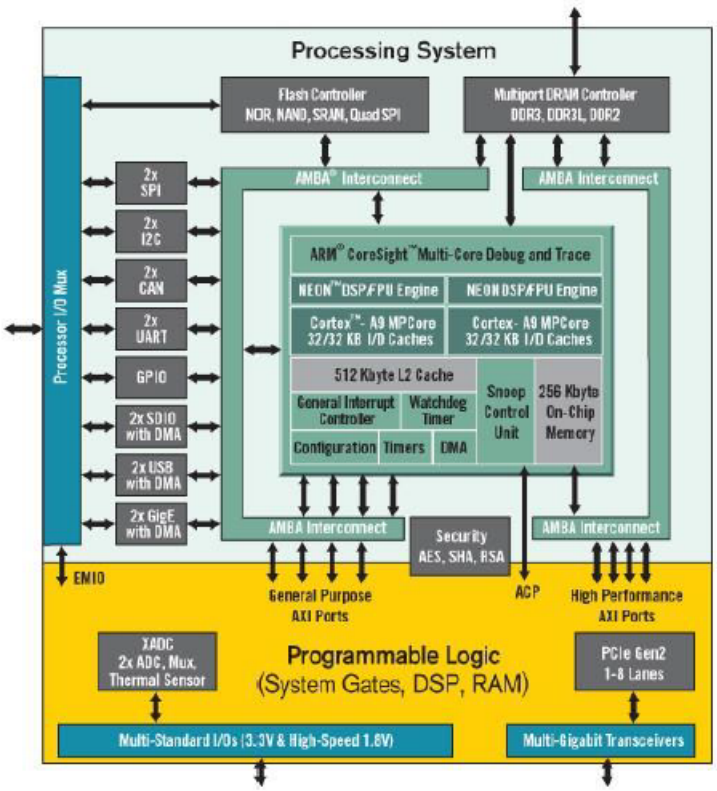


Figure 4.3 – l'Architecture AP SOC du Zynq[9]

Nous allons maintenant mettre la lumière sur les caractéristiques de chacune des parties de cet AP SOC

4.2.2.1 Présentation de la partie Logique programmable :

Comme il a été mentionné précédemment, la partie logique programmable est presque identique à une carte FPGA de la série Artix-7 de Xilinx. La famille Artix présente des caractéristiques intéressantes à savoir : 215 000 cellules logiques, des blocs RAM de taille 13 Mb, une interface mémoire dont la vitesse est 1.066 Mb/s, 500 pins E/S, une transmission full duplex avec un débit allant jusqu'à 211Gb/s etc... Ces caractéristiques rendent la possibilité de l'utilisation de cette partie intéressante car elles assurent une flexibilité et une multitude de choix dans la conception et l'implémentation.

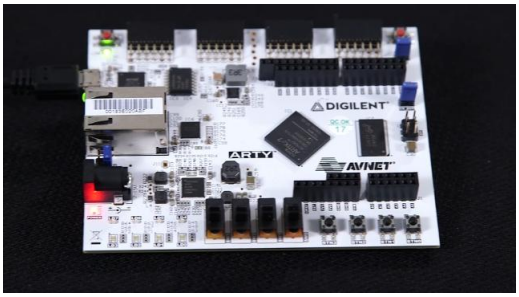


Figure 4.4 – La FPGA ARTY appartenant à la famille Artix-7

La différence réside dans deux points :
- Il existe des portes et des bus qui la relient à la partie ARM.
- Cette partie ne nécessite pas la configuration matérielle comme dans une FPGA typique. Elle peut être configurée à travers le processeur ou la porte JTAG.

4.2.2.2 Présentation de la partie processeur :

La partie processeur se compose de nombreux composants, y compris l'unité de traitement d'application (APU, qui comprend 2 Cortex-A9 processeurs), un bus maître/esclave de type Advanced Microcontroller Bus Architecture (AMBA), contrôleur de mémoire DDR3, et divers contrôleurs périphériques avec leurs entrées et sorties multiplexées à 54 broches dédiées (appelées Multiplexed I / O, ou des broches MIO). Les contrôleurs périphériques dont les E/S ne sont pas connectées aux broches MIO peuvent acheminer leur place I / O à travers la logique programmable, via l'interface Extended-MIO (EMIO). Les contrôleurs périphériques sont des esclaves reliés aux processeurs par l'intermédiaire du bus AMBA et contiennent des registres

de contrôle lecture/écriture qui sont adressables dans l'espace mémoire des processeurs. La logique programmable est également relié au bus comme un esclave, et les conceptions peuvent mettre en œuvre plusieurs cœurs dans le tissu de FPGA qui contiennent chacun aussi des registres de commande adressables. En outre, les noyaux mis en œuvre dans le PL peuvent déclencher des alarmes aux processeurs et d'effectuer DMA accède à la mémoire DDR3 [9]

Le processeur Cortex-A9 implémente l'ARMv7-A architecture qui comprend les éléments suivants :

• Un jeu d'instructions selon l'architecutre ARM Thumb®-2 32 bits pour la densité de code globale comparable avec Thumb et des performances comparables avec des instructions ARM

• Une Architecture Thumb Execution Environment (ThumbEE) pour l'accélération de l'environnement de l'exécution

• Extensions de sécurité pour une sécurité plus avancée.

• Extension de l'architecture avancée SIMD pour accélérer les performances des applications multimédia telles que les graphiques 3-D et le traitement de l'image.

• Vecteur virgule flottante version 3 (VFPv3) extension de l'architecture pour en virgule flottante calcul qui est entièrement conforme à la norme IEEE 754. [1]

Le processeur Cortex- A9 inclut les fonctionnalités suivantes :

• Un pipeline hors-du-commande, super scalaire, de longueur variable, avec la prédiction dynamique de branche

• Mise en œuvre intégrale de l'architecture ARM v7 – avec le jeu d'instructions

• Extensions de sécurité

• Un système de mémoire Harvard niveau 1 avec l'unité de gestion mémoire (MMU).

• Deux interfaces maître AXI 64 bits avec Master 0 pour le côté bus des données et Master 1 pour le côté bus d'instruction

• Un débogage selon L'architecture ARMv7

• Le support à la trace avec l'interface Program Trace Macrocell (PTM)

• Prise en charge de la gestion avancée de l'alimentation avec un maximum de 3 domaines d'alimentation

• Un moteur de préchargement optionnel

- L'accélération matérielle Jazelle optionnelle
- Moteur de données optionnel avec MPE et VFPv3. [1]

La figure ci-dessous montre l'architecture du processeur ARM Cortex A9

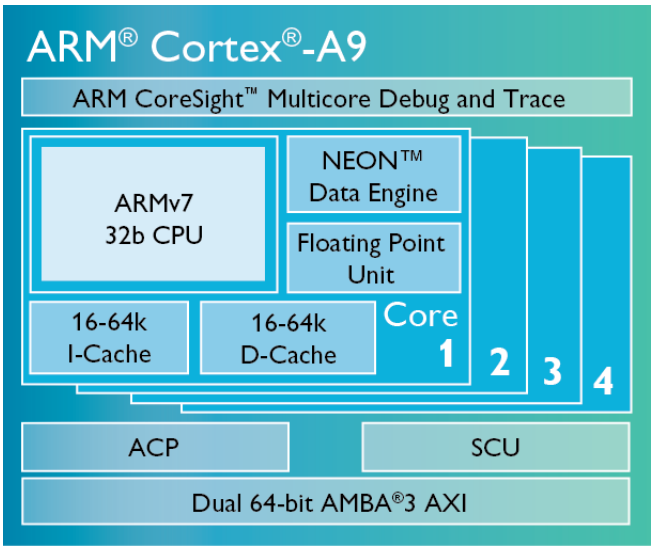


Figure 4.5 – L'architecture du Cortex-A9[2]

4.3 Implémentation embarquée sur la carte Zybo

Après avoir étudié les caractéristiques de notre environnement embarqué, nous allons maintenant procéder à l'implémentation de notre outil en 3D sur la carte Zybo Zynq.

Les étapes de l'implémentation embarquée sont décrites par ce processus :

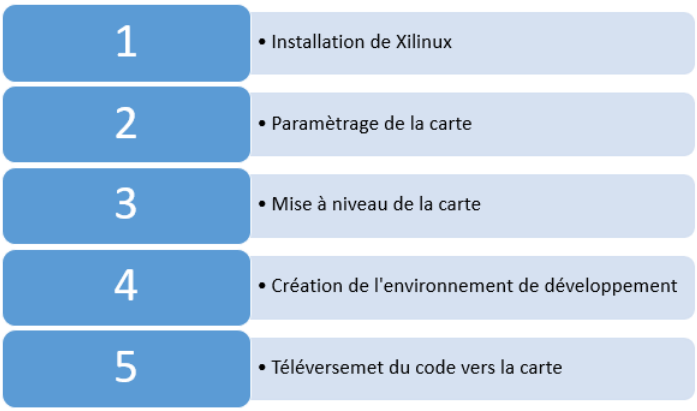


Figure 4.6 – Etapes de l'implémentation

Nous allons maintenant expliquer chacune des étapes de l'implémentation

4.3.1 Installation de Xilinux :

Xilinux est une distribution du système d'exploitation Ubuntu 12 produit de la communauté Xillybus. Cette distribution est dédiée aux cartes Zybo ainsi que toutes les cartes Z-board. Xilinux est basée sur la version Ubuntu 12.04 pour ARM, ce qui offre un environnement temps réel avec un ordonnanceur de tâches. Pour pouvoir installer Xilinux, nous avons besoin d'une carte SD qui va servir du disque dur pour la carte. Nous aurons besoin aussi, d'un clavier, une souris et un écran pour réaliser les manipulations de l'installation, Au final, nous avons besoin d'un câble Ethernet pour les manipulations SSH et les mises à jour. Côté software, nous allons utiliser Vivado, le logiciel qui génère les fichiers de la conception FPGA et les fichiers de configuration SSH au niveau du système.

Au début, nous commençons par la préparation des fichiers de la FPGA. Les fichiers sont classés en sept catégories :
- Des fichiers dédiés pour la programmation en VHDL
- Des fichiers dédiés pour la programmation en Verilog
- Des fichiers précompilés des « Xillybus IP cores »
- Des fichiers permettant la génération de tout ce qui est côté processeur
- Des fichiers pour la génération des fichiers globaux
- Les fichiers de démarrage de la FPGA
- Les fichiers de Vivado.

Après la préparation des fichiers, nous allons générer le fichier BitStream à l'aide de Vivado. Le fichier BitStream est le fichier généré après avoir implémenté tous les fichiers VHDL/Verilog sur la carte. Le résultat de cette implémentation sera un système hardware implémenté sur la carte.

Côté logiciel, nous allons partitionner notre carte SD en deux partitions : une première partition d'environs 16 Mo avec laquelle on peut démarrer Xilinux sur la carte, et une deuxième

qui contient tous les fichiers du système Linux.

4.3.2 Paramétrage de la carte :

Le paramétrage de la carte se fait sur deux volets : un volet hardware et un volet software :
- Volet hardware : Nous configurons les jumpers de la carte pour activer la sortie FPGA et nous connectons les périphériques USB, à savoir le souris, le clavier et l'écran, à travers un hub. Ensuite, nous injectons la carte SD et nous connectons le câble Ethernet. Tout ceci nous permettra de lancer le système d'exploitation Ubuntu sur la carte.
- Volet software : Après qu'on a pu démarrer Ubuntu sur la carte, nous créons un utilisateur avec un identifiant et un mot de passe pour accéder au système. Nous lançons l'interface graphique et nous activons le protocole SSH. Maintenant, on a notre système fonctionnel sur la carte. Néanmoins, il y a un problème côté taille mémoire. En effet, le système de fichier est trop gourmand en termes de mémoire. Donc, il faut redimensionner la carte SD afin de l'agrandir. Au final, nous allons mettre à jour tous les paquets du système pour pouvoir travailler après sans rencontrer les problèmes de compatibilité des paquets.

4.3.3 Mise à niveau de la carte :

Il s'agit du passage de la version 12 d'Ubuntu vers la version 14, la plus récente, pour assurer l'harmonie entre les paquets récents et le système de base.



Figure 4.7 – Logo d'Ubuntu

4.3.4 Création de l'environnement de développement :

Cette étape inclut l'installation des paquets spécifiques et nécessaires au développement et au fonctionnement de notre outil : les compilateurs gcc et g++, les débuggeurs gdb et valgrind, l'outil de visualisation graphique gnuplot, les différentes librairies pour la compilation du code...

4.3.5 Téléversement du code vers la carte :

Cette étape consiste à copier le code source de notre outil sur la carte SD, ou le disque dur de notre nouvel « ordinateur » et de l'exécuter.

4.4 Analyse des résultats :

A cause des limitations matérielles de la carte SD, nous avons choisi de travailler sur le terrain de l'Andorre. Sur le terrain de l'Andorre, nous avons fixé le pas entre les points de montée à 1000 m et l'angle de montée à 5.7 degrés, c'est-à-dire un arc de valeur 1500 m. Les différentes exécutions ont montré que lorsqu'on lance notre outil, la CPU de la carte est utilisée à son maximum. A partir de ça, nous pouvons estimer l'importance de l'outil côté jeu d'instructions et complexité algorithmique. Nous allons nous intéresser à l'impact du Radius Factor sur le temps d'exécution. Après, nous allons voir l'état de la mémoire au cours de l'exécution. Voir la mémoire alloué à l'outil va nous guider dans le choix du design du système à concevoir sur la FPGA.

4.4.1 Analyse côté temps d'exécution :

Nous nous sommes intéressés tout d'abord au temps d'exécution total afin d'estimer l'ordre de grandeur du temps. Cet indicateur va donner une idée sur la marge de temps à allouer dans les conditions de vol réels. Le tableau ci-dessous donne le temps global de l'exécution du software, avec la configuration mentionnée ci-dessus, en fonction du Radius Factor introduit en entrée :

Radius Factor	Temps global d'exécution (en s)
1.0	1
1.25	1.13
1.5	1.4
1.75	1.53
2	1.82
2.25	1.94
2.5	2.28
2.75	2.53
3	2.6
3.5	2.9
4	3.24
4.5	3.52
5	3.94
5.5	4.22
6	4.43
6.5	4.62
7	4.82

Table 4.2 – Temps d'exécution en fonction du Radius Factor

Nous constatons une évolution linéaire du temps d'exécution. Nous allons tracer l'allure de la courbe représentant l'évolution du temps d'exécution en fonction du Radius Factor :

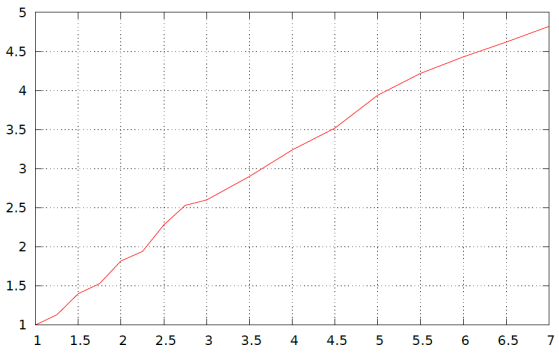


Figure 4.8 – Evoltuion du temps d'exécution en fonction du Radius Factor

Nous avons remarqué que, lorsque le Radius Factor dépasse la valeur du 2.25, la variation du temps d'exécution devient plus importante. Au niveau du chapitre précédent, nous avons conclu que l'outil passe la majorité du temps dans le calcul du graphe horizontal sur chaque palier. Nous allons maintenant vérifier que ces hypothèses restent valables sur la carte FPGA. Pour ce faire, nous avons collecté les temps des différentes phases de notre outil dans ce tableau.

Radius Factor	T1(ms)	T2(ms)	T3(ms)	T4(ms)	T5(ms)	T6(ms)
1.0	12.590	9.811	106.111	474.998	309.355	50.929
1.25	12.363	9.849	102.898	675.501	236.100	51.004
1.5	12.514	9.868	103.710	959.222	240.089	50.872
1.75	12.647	9.811	103.465	1075.543	240.807	51.061
2	12.439	9.811	102.916	1358.810	246.989	50.910
2.25	12.306	9.849	103.370	1481.445	247.254	50.966
2.5	12.420	9.849	104.051	1818.307	258.918	50.891
2.75	12.552	9.849	103.597	2062.007	252.717	50.929
3	12.344	9.849	103.257	2158.269	252.171	51.212
3.5	12.306	9.830	103.748	2436.773	253.851	51.363
4	12.571	9.868	103.332	2769.022	256.630	51.118
4.5	12.344	9.887	102.954	3077.603	255.836	50.872
5	12.495	9.849	103.654	3480.518	258.596	50.853
5.5	12.382	9.830	104.032	3769.533	259.768	51.193
6	12.477	9.849	102.973	3959.525	259.636	50.948
6.5	12.363	9.792	103.049	4152.579	259.882	51.023
7	12.401	9.849	103.597	65.469	260.063	51.042

Table 4.3 – Evolution des temps des différentes phases d'exécution

Les tests réalisés ont confirmé nos hypothèses. La phase de la génération du graphe horizontal total consomme énormément de temps. De plus, dès qu'on dépasse 2.25, sa variation devient très importante, 200 ms en moyenne, ce qui influe sur le temps d'exécution total de l'outil. Cette phase est la phase la plus gourmande de notre algorithme. Il faut donc lui allouer des ressources spécifiques pour son exécutions. Pour ce faire, nous pensons à exécuter cette phase

sur un ordinateur et laisser le reste sur la carte FPGA.

Dans le même contexte, Nous allons maintenant comparer le temps d'exécution total entre un PC portable et le processeur de la carte FPGA via cette courbe. Ceci nous donnera un aperçu sur la différence de l'ordre de grandeur des deux systèmes.

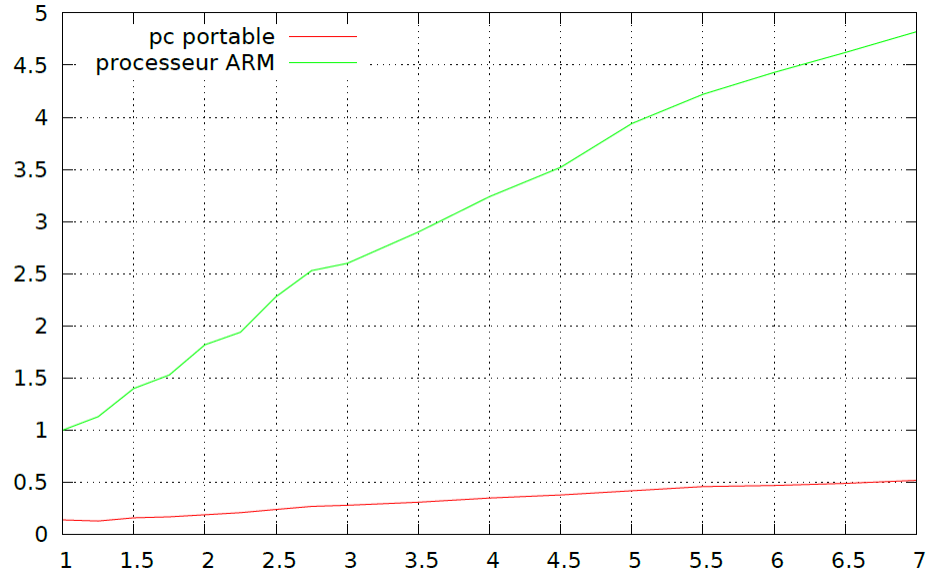


Figure 4.9 – Comparaison entre la FPGA et un ordinateur

Nous remarquons que le temps d'exécution sur un processeur est 10 fois plus grand que celui sur un ordinateur portable. Ceci s'explique par le fait que la fréquence d'un processeur d'un ordinateur est beaucoup plus importante que celle d'un processeur ARM. En effet, nous avons testé l'outil sur un ordinateur doté d'un processeur Intel i7 dont la fréquence estr 2.5 GHz alors que le Cortex A9 est à 650 MHz, soit un ratio de fréquence de 3.85. Il est donc préférable d'exécuter la partie génération du graphe sur l'ordinateur et laisser le reste des étapes sur la carte.

4.4.2 Analyse côté mémoire :

Nous avons collecté les valeurs de la quantité mémoire alloué aux données lors de chaque exécution. Les résultats sont présentés dans le tableau ci-dessous :

Radius Factor	Data	Data	Data	Data	Data	Data
1.0	64533	64533	61603	61603	79	79
1.25	64533	64533	61603	61603	79	79
1.5	64533	64533	61603	61603	79	79
1.75	64533	64533	61603	61603	79	79
2	64533	64533	61603	61603	79	79
2.25	64533	64533	61603	61603	79	79
2.5	64533	64533	61603	61603	79	79
2.75	64533	64533	61603	61603	79	79
3	64533	64533	61603	61603	79	79
3.5	64533	64533	61603	61603	79	79
4	64533	64533	61603	61603	79	79
4.5	64533	64533	61603	61603	79	79
5	64533	64533	61603	61603	79	79
5.5	64533	64533	61603	61603	79	79
6	64533	64533	61603	61603	79	79
6.5	64533	64533	61603	61603	79	79
7	64533	64533	61603	61603	79	79

Table 4.4 – Evolution de la quantité de mémoire allouée aux données

Nous remarquons que la variation a la même allure que celle retrouvée lorsqu'on a testé la version 3D sur le terrain des Pyrénées sur un ordinateur portable. On peut donc déduire que l'implémentation sur la carte Zybo n'a pas affecté les performances mémoire de notre outil. Cette constatation peut être appuyée par le fait que l'allocation mémoire ne s'affecte pas par le changement de l'environnement. Elle s'affecte seulement de la taille de la RAM et du disque dur. Par exemple, si notre carte SD était plus petite, on aura sûrement des problèmes au niveau de l'exécution de l'outil.

4.5 Conclusion :

Au cours de ce chapitre, nous avons pu implémenter notre outil software sur une carte CPU/FPGA dotée d'un système d'exploitation Ubuntu. Nous avons finalisé notre travail

par une analyse des résultats collectés de la carte. Les résultats ont confirmé les résultats récupérés dans les deux chapitres précédents. En effet, nous avons constaté que notre outil est gourmand en terme de temps et de mémoire. Même après avoir construit la version 3D sur la version 2D, nous avons besoin d'une méthode d'optimisation plus efficace afin de réduire encore plus les coûts. Avec cette analyse, nous avons parvenu à terminer le travail demandé et à résoudre les trois problématiques clés du projet.

CONCLUSION GENERALE

Le but de notre travail était le développement et l'implémentation embarquée des algorithmes de plan des vols pour aéronefs. Ce travail répond à un besoin crucial chez Airbus Helicopters. Il s'agit de mettre en place un outil software capable de calculer un graphe dans une zone précise de l'espace et de l'implémenter dans une carte embarqué.

Notre travail était essentiellement orienté vers l'optimisation de la version existante du projet après une étude de sa complexité temporelle et spatiale en guise d'améliorer ses performances. Nous nous sommes après focalisés sur la migration de la version 2D vers une version 3D qui permet de générer un graphe complet dans l'espace. Cette migration a été faite par le développement et le déploiement d'un module de discrétisation et de calcul d'un graphe tridimensionnel. Ce graphe sera utilisé après comme une entrée pour l'algorithme A* afin de déterminer le chemin optimal entre deux points. Finalement, nous avons assuré l'implémentation de notre outil dans un environnement embarqué temps réel basée sur une carte FPGA et un système d'exploitation Linux. Nous avons travaillé dans un environnement basé sur le système d'exploitation Debian, connu par ses spécificités temps réel et ses applications dans le domaine aéronautique. Notre travail était basé sur le langage C/C++ et les outils de compilation et débogage relatifs.

Comme perspectives, nous envisageons l'intégration de l'algorithme A* dans la version 3D afin de calculer le chemin optimal entre deux points. Nous envisageons aussi la parallélisation partielle ou complète de l'outil sur la FPGA afin de réduire le temps d'exécution. De plus, nous pensons à introduire la phase dynamique de l'outil. Cette phase permet la replanification de la mission en temps réel en prenant en compte les obstacles et les contraintes de l'environnement.

Bibliographie

[1] Cortex A9 Technical Reference Manual.

[2] Cortex-a9 processor. URL http://www.arm.com/products/processors/cortex-a/cortex-a9.php.

[3] Smart move : Intelligent path-finding. URL http://www.gamasutra.com/view/feature/131724/smart_move_intelligent_.php?page=4.

[4] Quelques dates clés. URL https://www.univ-valenciennes.fr/quelques-dates-clefs.

[5] Histoire de l'uvhc. URL https://www.univ-valenciennes.fr/notre-histoire.

[6] Présentation du lamih. URL http://www.univ-valenciennes.fr/LAMIH/fr/presentation.

[7] Organigramme du lamih. URL http://www.univ-valenciennes.fr/LAMIH/fr/organigramme.

[8] effective earth-radius factor. URL http://www.electropedia.org/iev/iev.nsf/display?openform&ievref=705-05-41.

[9] Zybo Reference Manual.

[10] M. L. Alex Nash, Sven Koenig. Incremental phi* : Incremental any-angle path planning on grids.

[11] S. Chen. The a* search algorithm.

[12] A. S. Dave Ferguson. Field d* : An interpolation-based path planner and replanner.

[13] M. V. H. Radha, R. Leonardi and B. Naylor. Binary space partitioning (bsp) tree representation of images. Journal of Visual Communication and Image Reprsentation, September 1991.

[14] R. K. Hamid R.Rabiee and S. R. Safavian. Multiresolution segmentation-based image coding with hierarchical structures.

[15] A. K. Hanan Samet. Octree approximation and compression methods.

[16] H.Samet. Application of spatial data structures : Computer graphics, image processing and gis. Reading, MA, 1990.

[17] C. C. M. C. S. Inc. Les Quadtrees. 2008.

[18] S. K. A. F. Kenny Daniel, Alex Nash. Theta* : Any-angle path planning on grids. Journal of Artificial Intelligence Research, October 2010.

[19] C.-F. M. Michel DHOME, CNRS. Évaluation de l'AERES sur l'unité :Laboratoire d'Automatique, de Mécanique et d'Informatique Industrielles et Humaines LAMIH. 2013.

[20] D. F. Maxom Likhachev. Lifelong planning a*. ELSEVIER, April 2002.

[21] H. Samet. An overview of quadtrees, octrees and related hierarchical data structures.

[22] A. Stentz. The foucussed d* alogrithm for real-time replanning.

[23] A. Stentz. Optimal and efficient path planning for unknown and dynamique environnements. August 1993.

[24] M. L. Sven Koenig. D* lite.

[25] J. Vaisey and A. Gersho. Image compression with variable block size segmentation. IEEE Trans. Signal Processing.